

Texture3D subresource tiling

Article • 10/20/2022

This table shows how [Texture3D](#) subresources are tiled. The values in this table don't count tail mip packing.

This table takes the [Texture2D](#) tiling and divides the x/y dimensions by 4 each and adds 16 layers of depth. All the tiles for the first plane (2D plane of tiles defining the first 16 layers of depth) appear before the subsequent planes.

Note [Texture3D](#) support in streaming resources isn't exposed in the initial implementation of streaming resources, but the desired tile shapes are listed here for possible support in a future release.

Bits/Pixel (1 sample/pixel)	Tile Dimensions (Pixels, WxHxD)
8	64x32x32
16	32x32x32
32	32x32x16
64	32x16x16
128	16x16x16
BC1,4	128x64x16
BC2,3,5,6,7	64x64x16

Format bit counts not supported with streaming resources are 96 bpp formats, video formats, DXGI_FORMAT_R1_UNORM, DXGI_FORMAT_R8G8_B8G8_UNORM, and DXGI_FORMAT_R8R8_G8B8_UNORM.

Related topics

[How a streaming resource's area is tiled](#)

Buffer tiling

Article • 10/20/2022

A [Buffer](#) resource is divided into 64KB tiles, with some empty space in the last tile if the size is not a multiple of 64KB.

Structured buffers must have no constraint on the stride to be tiled. But possible performance optimizations in hardware for using [StructuredBuffers](#) can be sacrificed by making them tiled in the first place.

Related topics

[How a streaming resource's area is tiled](#)

Mipmap packing

Article • 10/20/2022

Some number of mips (per array slice) can be packed into some number of tiles, depending on a streaming resource's dimensions, format, number of mipmaps, and array slices.

Depending on the [tier](#) of streaming resources support, mipmaps with certain dimensions don't follow the standard tile shapes and are considered to all be packed together with one another in a manner that is opaque to the application. Higher tiers of support have broader guarantees about what types of surface dimensions fit in the standard tile shapes (and can therefore be individually mapped by applications).

What can vary between implementations is that—given a streaming resource's dimensions, format, number of mipmaps, and array slices—some number M of mips (per array slice) can be packed into some number N tiles. When you get the resource tiling information for a device, the driver reports to the application what M and N are (among other details about the surface that are standard and don't vary by hardware vendor). The set of tiles for the packed mips are still 64KB and can be individually mapped into disparate locations in a tile pool.

But the pixel shape of the tiles and how the mipmaps fit across the set of tiles is specific to a hardware vendor and too complex to expose. So, applications are required to either map all of the tiles that are designated as packed, or none of them, at a time. Otherwise, the behavior for accessing the streaming resource is undefined.

For arrayed surfaces, the set of packed mips and the number of packed tiles storing those mips (M and N described preceding) applies individually for each array slice.

Dedicated APIs for copying tiles can't access packed mips. Applications that want to copy data to and from packed mips can do so using all the non-streaming resource-specific APIs for copying and rendering to surfaces.

Related topics

[How a streaming resource's area is tiled](#)

Pipeline access to streaming resources

Article • 10/20/2022

Streaming resources can be used in shader resource views (SRV), render target views (RTV), depth stencil views (DSV) and unordered access views (UAV), as well as some bind points where views aren't used, such as vertex buffer bindings. For the list of supported bindings, see [Streaming resource creation parameters](#). The various D3D Copy operations also work on streaming resources.

If multiple tile coordinates in one or more views is bound to the same memory location, reads and writes from different paths to the same memory will occur in a non-deterministic and non-repeatable order of memory accesses.

If all tiles behind a memory access footprint from a shader are mapped to unique tiles, behavior is identical on all implementations to the surface having the same memory contents in a non-tiled fashion.

In this section

Topic	Description
SRV behavior with non-mapped tiles	Behavior of shader resource view (SRV) reads that involve non-mapped tiles depends on the level of hardware support.
UAV behavior with non-mapped tiles	Behavior of unordered access view (UAV) reads and writes depends on the level of hardware support.
Rasterizer behavior with non-mapped tiles	This section describes rasterizer behavior with non-mapped tiles.
Tile access limitations with duplicate mappings	There are limitations on tile access with duplicate mappings, such as when copying streaming resources with overlapping source and destination, or when rendering to tiles shared within the render area.

Topic	Description
Streaming resources texture sampling features	Streaming resources texture sampling features include getting shader status feedback about mapped areas, checking whether all data being accessed was mapped in the resource, clamping to help shaders avoid areas in mipmapped streaming resources that are known to be non-mapped, and discovering what the minimum LOD that is fully mapped for an entire texture filter footprint will be.
HLSL streaming resources exposure	A specific Microsoft High Level Shader Language (HLSL) syntax is required to support streaming resources in Shader Model 5 .

Related topics

[Streaming resources](#)

SRV behavior with non-mapped tiles

Article • 10/20/2022

Behavior of shader resource view (SRV) reads that involve non-mapped tiles depends on the level of hardware support. For a breakdown of requirements, see [read behavior for Streaming resources features tiers](#). This section summarizes the ideal behavior, which [Tier 2](#) requires.

Consider a texture filter operation that reads from a set of texels in an SRV. Texels that fall on non-mapped tiles contribute 0 in all non-missing components of the format (and the default for missing components) into the overall filter operation alongside contributions from mapped texels. The texels are all weighted and combined together independent of whether the data came from mapped or non-mapped tiles.

Some first-generation [Tier 2](#) level hardware does not meet this spec requirement, and returns the 0 with defaults described preceding as the overall filter result if any texels (with nonzero weight) fall on non-mapped tiles. No other hardware will be allowed to miss the requirement to include all (nonzero weight) texels in the filter.

Related topics

[Pipeline access to streaming resources](#)

UAV behavior with non-mapped tiles

Article • 10/20/2022

Behavior of unordered access view (UAV) reads and writes depends on the level of hardware support. For a breakdown of requirements, see overall read and write behavior for [Streaming resources features tiers](#). This section summarizes the ideal behavior.

Shader operations that read from a non-mapped tile in a UAV return 0 in all non-missing components of the format, and the default for missing components.

Shader operations that attempt to write to a non-mapped tile cause nothing to be written to the non-mapped area (while writes to a mapped area proceed). This ideal definition for write handling is not required by [Tier 2](#); writes to non-mapped tiles can end up in a cache that subsequent reads could pick up.

Related topics

[Pipeline access to streaming resources](#)

Rasterizer behavior with non-mapped tiles

Article • 10/20/2022

This section describes rasterizer behavior with non-mapped tiles.

DepthStencilView

Behavior of depth stencil view (DSV) reads and writes depends on the level of hardware support. For a breakdown of requirements, see overall read and write behavior for [Streaming resources features tiers](#).

Here is the ideal behavior:

If a tile isn't mapped in the DepthStencilView, the return value from reading depth is 0, which is then fed into whatever operations are configured for the depth read value. Writes to the missing depth tile are dropped. This ideal definition for write handling is not required by [Tier 2](#); writes to non-mapped tiles can end up in a cache that subsequent reads could pick up.

RenderTargetView

Behavior of render target view (RTV) reads and writes depends on the level of hardware support. For a breakdown of requirements, see overall read and write behavior for [Streaming resources features tiers](#).

On all implementations, different RTVs (and DSV) bound simultaneously can have different areas mapped versus non-mapped and can have different sized surface formats (which means different tile shapes).

Here is the ideal behavior:

Reads from RTVs return 0 in missing tiles and writes are dropped. This ideal definition for write handling is not required by [Tier 2](#); writes to non-mapped tiles can end up in a cache that subsequent reads could pick up.

Related topics

[Pipeline access to streaming resources](#)

Tile access limitations with duplicate mappings

Article • 10/20/2022

There are limitations on tile access with duplicate mappings, such as when copying streaming resources with overlapping source and destination, or when rendering to tiles shared within the render area.

Copying streaming resources with overlapping source and destination

If tiles in the source and destination area of a Copy* operation have duplicated mappings in the copy area that would have overlapped even if both resources were not streaming resources and the Copy* operation supports overlapping copies, the Copy* operation will behave fine (as if the source is copied to a temporary location before going to the destination). But if the overlap is not obvious (like the source and destination resources are different but share mappings or mappings are duplicated over a given surface), results of the copy operation on the tiles that are shared are undefined.

Copying to streaming resource with duplicated tiles in destination area

Copying to a streaming resource with duplicated tiles in the destination area produces undefined results in these tiles unless the data itself is identical; different tiles might write the tiles in different orders.

UAV accesses to duplicate tiles mappings

Suppose an unordered access view (UAV) on a streaming resource has duplicate tile mappings in its area or with other resources bound to the pipeline. Ordering of accesses to these duplicated tiles is undefined if performed by different threads, just as any ordering of memory access to UAVs in general is unordered.

Rendering after tile mapping changes or content updates from outside mappings

If a streaming resource's tile mappings have changed or content in mapped tiled pool tiles have changed via another streaming resource's mappings, and the streaming resource is going to be rendered via render target view or depth stencil view, the application must Clear (using the fixed function Clear APIs) or fully copy over using Copy*/Update* APIs the tiles that have changed within the area being rendered (mapped or not).

Failure of an application to clear or copy in these cases results in hardware optimization structures for the given render target view or depth stencil view being stale, and will result in garbage rendering results on some hardware and inconsistency across different hardware. These hidden optimization data structures used by hardware might be local to individual mappings and not visible to other mappings to the same memory.

When you clear a resource view (setting all the elements in a resource view to one value), you can clear render target views with rectangles. For hardware that supports streaming resources, clearing a resource view must also support clearing of depth stencil views with rectangles, for depth only surfaces (without stencil). This operation allows applications to clear only the necessary area of a surface.

If an application needs to preserve existing memory contents of areas in a streaming resource where mappings have changed, that application must work around the clear requirement. The application can accomplish this work-around by first saving the contents where tile mappings have changed (by copying them to a temporary surface), issuing the required clear command, and then copying the contents back. While this would accomplish the task of preserving surface contents for incremental rendering, the downside is that subsequent rendering performance on the surface might suffer because rendering optimizations might be lost.

If a tile is mapped into multiple streaming resources at the same time and tile contents are manipulated by any means (render, copy, and so on) via one of the streaming resources, if the same tile is to be rendered via any other streaming resource, the tile must be cleared first as previously described.

Rendering to tiles shared outside render area

Suppose an area in a streaming resource is being rendered to and the tile pool tiles referenced by the render area are also mapped to from outside the render area (including via other streaming resources, at the same time or not). Data rendered to these tiles isn't guaranteed to appear correctly when viewed through the other mappings, even though the underlying memory layout is compatible. This fact is due to optimization data structures some hardware use that can be local to individual

mappings for renderable surfaces and not visible to other mappings to the same memory location.

You can work around this restriction by copying from the rendered mapping to all the other mappings to the same memory that might be accessed (or clearing that memory or copying other data to it if the old contents are no longer needed). While this work-around seems redundant, it makes all other mappings to the same memory correctly understand how to access its contents, and at least the memory savings of having only a single physical memory backing remains intact.

Also, when you switch between using different streaming resources that share mappings (unless only reading), you must specify a data access ordering constraint (a barrier) between multiple tiled resources, in between the switches.

Rendering to tiles shared within render area

If an area in a streaming resource is being rendered to and within the render area multiple tiles are mapped to the same tile pool location, rendering results are undefined on those tiles.

Data compatibility across streaming resources sharing tiles

Suppose multiple streaming resources have mappings to the same tile pool locations and each resource is used to access the same data. This scenario is only valid if the other rules about avoiding problems with hardware optimization structures are avoided, appropriate barriers are specified (specifying a data access ordering constraint between multiple tiled resources), and the streaming resources are compatible with each other.

The latter is described here in terms of what it means for streaming resources sharing tiles to be incompatible. The incompatibility conditions of accessing the same data across duplicate tile mappings are the use of different surface dimensions or format, or differences in the presence of render target or depth stencil bind flags on the resources. Writing to the memory with one type of mapping produces undefined results if you subsequently read or render via a mapping from an incompatible resource.

If the other resource sharing mappings are first initialized with new data (recycling the memory for a different purpose), the subsequent read or render operation is fine since data isn't bleeding across incompatible interpretations. But, when you switch between accessing incompatible mappings like this, you must specify barriers (specifying a data access ordering constraint between multiple tiled resources).

If the render target or depth stencil bind flag isn't set on any of the resources sharing mappings with each other, there are far fewer restrictions. As long as the format and surface types (for example, Texture2D) are the same, tiles can be shared. Different formats being compatible are cases such as BC* surfaces and the equivalent sized uncompressed 32 bit or 16 bit per component format, like BC6H and R32G32B32A32. Many 32 bit per element formats can be aliased with R32_* as well (R10G10B10A2_*, R8G8B8A8_*, B8G8R8A8_*, B8G8R8X8_*, R16G16_*); this operation has always been allowed for non-streaming resources.

Sharing between packed and non-packed tiles is fine if the formats are compatible and the tiles are filled with solid color.

Finally, if nothing is common about the resources sharing tile mappings except that none have render target or depth stencil bind flags, only memory filled with 0 can be shared safely; the mapping will appear as whatever 0 decodes to for the definition of the given resource format (typically just 0).

Related topics

[Pipeline access to streaming resources](#)

Streaming resources texture sampling features

Article • 10/20/2022

Streaming resources texture sampling features include getting shader status feedback about mapped areas, checking whether all data being accessed was mapped in the resource, clamping to help shaders avoid areas in mipmapped streaming resources that are known to be non-mapped, and discovering what the minimum LOD that is fully mapped for an entire texture filter footprint will be.

Requirements of streaming resources texture sampling features

The texture sampling features described here require [Tier 2](#) level of streaming resources support.

Shader status feedback about mapped areas

Any shader instruction that reads and/or writes to a streaming resource causes status information to be recorded. This status is exposed as an optional extra return value on every resource access instruction that goes into a 32-bit temp register. The contents of the return value are opaque. That is, direct reading by the shader program is disallowed. But, you can use the [CheckAccessFullyMapped](#) function to extract the status info.

Fully mapped check

The [CheckAccessFullyMapped](#) function interprets the status returned from a memory access and indicates whether all data being accessed was mapped in the resource. **CheckAccessFullyMapped** returns true (0xFFFFFFFF) if data was mapped or false (0x00000000) if data was unmapped.

During filter operations, sometimes the weight of a given texel ends up being 0.0. An example is a linear sample with texture coordinates that fall directly on a texel center: 3 other texels (which ones they are can vary by hardware) contribute to the filter but with 0 weight. These 0 weight texels don't contribute to the filter result at all, so if they happen to fall on **NULL** tiles, they don't count as an unmapped access. Note the same guarantee applies for texture filters that include multiple mip levels; if the texels on one

of the mipmaps isn't mapped but the weight on those texels is 0, those texels don't count as an unmapped access.

When sampling from a format that has fewer than 4 components (such as `DXGI_FORMAT_R8_UNORM`), any texels that fall on **NULL** tiles result in a **NULL** mapped access being reported, regardless of which components the shader actually looks at in the result. For example, reading from `R8_UNORM` and masking the read result in the shader with `.gba/.yzw` wouldn't appear to need to read the texture at all. But if the texel address is a **NULL** mapped tile, the operation still counts as a **NULL** map access.

The shader can check the status and pursue any desired course of action on failure. For example, a course of action can be logging 'misses' (say via UAV write) and/or issuing another read clamped to a coarser LOD known to be mapped. An application might want to track successful accesses as well in order to get a sense of what portion of the mapped set of tiles got accessed.

One complication for logging is no mechanism exists for reporting the exact set of tiles that would have been accessed. The application can make conservative guesses based on knowing the coordinates it used for access, as well as using the LOD instruction; for example, `tex2Dlod` returns the hardware LOD calculation.

Another complication is that lots of accesses will be to the same tiles, so a lot of redundant logging will occur and possibly contention on memory. It could be convenient if the hardware could be given the option to not bother to report tile accesses if they were reported elsewhere before. Perhaps the state of such tracking could be reset from the API (likely at frame boundaries).

Per-sample MinLOD clamp

To help shaders avoid areas in mipmapped streaming resources that are known to be non-mapped, most shader instructions that involve using a sampler (filtering) have a mode that allows the shader to pass an additional float32 MinLOD clamp parameter to the texture sample. This value is in the view's mipmap number space, as opposed to the underlying resource.

The hardware performs `max(fShaderMinLODClamp, fComputedLOD)` in the same place in the LOD calculation where the per-resource MinLOD clamp occurs, which is also a `max()`.

If the result of applying the per-sample LOD clamp and any other LOD clamps defined in the sampler is an empty set, the result is the same out of bounds access result as the per-resource minLOD clamp: 0 for components in the surface format and defaults for missing components.

The LOD instruction (for example, [tex2Dlod](#)), which predates the per-sample minLOD clamp described here, returns both a clamped and unclamped LOD. The clamped LOD returned from this LOD instruction reflects all clamping including the per-resource clamp, but not a per-sample clamp. Per-sample clamp is controlled and known by the shader anyway, so the shader author can manually apply that clamp to the LOD instruction's return value if desired.

Min/Max reduction filtering

Applications can choose to manage their own data structures that inform them of what the mappings looks like for a streaming resource. An example would be a surface that contains a texel to hold information for every tile in a streaming resource. One might store the first LOD that is mapped at a given tile location. By careful sampling of this data structure in a similar way that the streaming resource is intended to be sampled, one might discover what the minimum LOD that is fully mapped for an entire texture filter footprint will be. To help make this process easier, Direct3D 11.2 introduces a new general purpose sampler mode, min/max filtering.

The utility of min/max filtering for LOD tracking might be useful for other purposes, such as, perhaps the filtering of depth surfaces.

Min/max reduction filtering is a mode on samplers that fetches the same set of texels that a normal texture filter would fetch. But instead of blending the values to produce an answer, it returns the `min()` or `max()` of the texels fetched, on a per-component basis (for example, the min of all the R values, separately from the min of all the G values and so on).

The min/max operations follow Direct3D arithmetic precision rules. The order of comparisons doesn't matter.

During filter operations that aren't min/max, sometimes the weight of a given texel ends up being 0.0. An example is a linear sample with texture coordinates that fall directly on a texel center; in that case, 3 other texels (which ones they are may vary by hardware) contribute to the filter, but with 0 weight. For any of these texels that would be 0 weight on a non-min/max filter, if the filter is min/max, these texels still do not contribute to the result (and the weights do not otherwise affect the min/max filter operation).

Support for this feature depends on [Tier 2](#) support for streaming resources.

Related topics

[Pipeline access to streaming resources](#)

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

HLSL streaming resources exposure

Article • 10/20/2022

A specific Microsoft High Level Shader Language (HLSL) syntax is required to support streaming resources in [Shader Model 5](#).

The HLSL syntax for Shader Model 5 is allowed only on devices with streaming resources support. Each relevant HLSL method for streaming resources in the following table accepts either one (feedback) or two (clamp and feedback in this order) additional optional parameters. For example, a **Sample** method is:

Sample(sampler, location [, offset [, clamp [, feedback]]])

An example of a **Sample** method is [Texture2D.Sample\(S,float,int,float,uint\)](#).

The offset, clamp and feedback parameters are optional. You must specify all optional parameters up to the one you need, which is consistent with the C++ rules for default function arguments. For example, if the feedback status is needed, both offset and clamp parameters need to be explicitly supplied to **Sample**, even though they may not be logically needed.

The clamp parameter is a scalar float value. The literal value of clamp=0.0f indicates that the clamp operation isn't performed.

The feedback parameter is a **uint** variable that you can supply to the memory-access querying intrinsic [CheckAccessFullyMapped](#) function. You must not modify or interpret the value of the feedback parameter; but, the compiler doesn't provide any advanced analysis and diagnostics to detect whether you modified the value.

Here is the syntax of [CheckAccessFullyMapped](#):

bool CheckAccessFullyMapped(in uint FeedbackVar);

[CheckAccessFullyMapped](#) interprets the value of *FeedbackVar* and returns true if all data being accessed was mapped in the resource; otherwise, **CheckAccessFullyMapped** returns false.

If either the clamp or feedback parameter is present, the compiler emits a variant of the basic instruction. For example, sample of a streaming resource generates the `sample_cl_s` instruction.

If neither clamp nor feedback is specified, the compiler emits the basic instruction, so that there is no change from the current behavior.

The clamp value of 0.0f indicates that no clamp is performed; thus, the driver compiler can further tailor the instruction to the target hardware. If feedback is a NULL register in an instruction, the feedback is unused; thus, the driver compiler can further tailor the instruction to the target architecture.

If the HLSL compiler infers that clamp is 0.0f and feedback is unused, the compiler emits the corresponding basic instruction (for example, `sample` rather than `sample_cl_s`).

If a streaming resource access consists of several constituent byte code instructions, for example, for structured resources, the compiler aggregates individual feedback values via the OR operation to produce the final feedback value. Therefore, you see a single feedback value for such a complex access.

This is the summary table of HLSL methods that are changed to support feedback and/or clamp. These all work on tiled and non-streaming resources of all dimensions. Non-streaming resources always appear to be fully mapped.

HLSL objects	Intrinsic methods with feedback option (*) - also has clamp option
[RW]Texture2D	Gather
[RW]Texture2DArray	GatherRed
TextureCUBE	GatherGreen
TextureCUBEArray	GatherBlue
	GatherAlpha
	GatherCmp
	GatherCmpRed
	GatherCmpGreen
	GatherCmpBlue
	GatherCmpAlpha

HLSL objects	Intrinsic methods with feedback option (*) - also has clamp option
[RW]Texture1D	Sample*
[RW]Texture1DArray	SampleBias*
[RW]Texture2D	SampleCmp*
[RW]Texture2DArray	SampleCmpLevelZero
[RW]Texture3D	SampleGrad*
TextureCUBE	SampleLevel
TextureCUBEArray	
[RW]Texture1D	Load
[RW]Texture1DArray	
[RW]Texture2D	
Texture2DMS	
[RW]Texture2DArray	
Texture2DArrayMS	
[RW]Texture3D	
[RW]Buffer	
[RW]ByteAddressBuffer	
[RW]StructuredBuffer	

Related topics

[Pipeline access to streaming resources](#)

Streaming resources features tiers

Article • 10/20/2022

Direct3D supports streaming resources in three tiers of capabilities.

Tier 1 provides basic capabilities for streaming resources.

Tier 2 adds capabilities beyond Tier 1, such as guaranteeing non-packed texture mipmap when the size is at least one standard tile shape; shader instructions for clamping level-of-detail (LOD) and for obtaining status about the shader operation; also, reading from NULL-mapped tiles treat that sampled value as zero.

Tier 3 adds Texture3D capabilities, beyond Tier 2.

Query functions are available in the versions of Direct3D, to validate hardware and driver support for streaming resources, and at what tier level.

In this section

Topic	Description
Tier 1	This section describes tier 1 support.
Tier 2	Tier 2 support for streaming resources adds capabilities beyond Tier 1, such as guaranteeing nonpacked texture mipmap when the size is at least one standard tile shape; shader instructions for clamping level-of-detail (LOD) and for obtaining status about the shader operation; also, reading from NULL-mapped tiles treat that sampled value as zero.
Tier 3	Tier 3 adds support for Texture3D for streaming resources, in addition to the Tier 2 capabilities.

Related topics

[Streaming resources](#)

Tier 1

Article • 10/20/2022

This section describes tier 1 support.

Tier 1 general limitations

- Hardware at feature level 11.0 minimum.
- No quilting support.
- No Texture1D or Texture3D support.
- No 2, 8 or 16 sample multisample antialiasing (MSAA) support. Only 4x is required, except no 128 bpp formats.
- No standard swizzle pattern (layout within 64KB tiles and tail mip packing is up to the hardware vendor).
- Limitations on how tiles can be accessed when there are duplicate mappings. See [Tile access limitations with duplicate mappings](#).

Specific limitations affecting tier 1 only

Reading/writing to streaming resources that have NULL mappings

Streaming resources can have **NULL** mappings but reading from them or writing to them produces undefined results, including device removed. Applications can get around this by mapping a single dummy page to all the empty areas. Take care if you write and render to a page that is mapped to multiple render target locations, because the order of writes will be undefined.

No shader instructions for clamping LOD and mapped status feedback

Shader instructions for clamping LOD and mapped status feedback are not available. See [HLSL streaming resources exposure](#).

Alignment constraints for standard tile shapes

It is only guaranteed that mips (starting from the finest) whose dimensions are all multiples of the standard tile size support the standard tile shapes and can have

individual tiles arbitrarily mapped/unmapped. The first mipmap in a streaming resource that has any dimension not a multiple of standard tile size, along with all coarser mipmaps, can have a non-standard tiling shape, fitting into N 64KB tiles for this set of mips at once (N reported to the application). These N tiles are considered packed as one unit, which must be either fully mapped or fully unmapped by the application at any given time, though the mappings of each of the N tiles can be at arbitrarily disjoint locations in a tile pool.

Array of mipmaps that aren't a multiple of standard tile size

Streaming resources with any mipmaps not a multiple of standard tile size in all dimensions are not allowed to have an array size larger than 1.

Switching between referencing tiles in a tile pool via a Buffer and Texture resource

In order to switch between referencing tiles in a tile pool via a [Buffer](#) resource to referencing the same tiles via a [Texture](#) resource, or vice-versa, the most recent updating of tile mappings or copying of tile mappings that defines mappings to those tile pool tiles must be for the same resource dimension (Buffer versus Texture*) as the resource dimension that will be used to access the tiles. Otherwise, behavior is undefined, including the chance of device reset.

So, for example, it is invalid to update the tile mappings to define tile mappings for a Buffer, then update the tile mappings to the same tiles in the tile pool via a [Texture2D](#) resource, then access the tiles via the Buffer. Work-around operations are to either redefine tile mappings for a resource when switching between Buffer and Texture (or vice versa) sharing tiles or just never sharing tiles in a tile pool between Buffer resources and Texture resources.

Min/Max reduction filtering

Min/Max reduction filtering is not supported. See [Streaming resources texture sampling features](#).

Related topics

[Streaming resources features tiers](#)

Tier 2

Article • 10/20/2022

Tier 2 support for streaming resources adds capabilities beyond Tier 1, such as guaranteeing nonpacked texture mipmap when the size is at least one standard tile shape; shader instructions for clamping level-of-detail (LOD) and for obtaining status about the shader operation; also, reading from NULL-mapped tiles treat that sampled value as zero.

Tier 2 general support

Tier 2 support includes the following.

- Hardware at Feature Level 11.1 minimum.
- All features of the previous tier (without [Tier 1](#) specific limitations) plus the additions in these following items:
- Shader instructions for clamping LOD and mapped status feedback are available. See [HLSL streaming resources exposure](#).

In addition to these, there are some specific support issues that follow.

Non-mapped tiles

Reads from non-mapped tiles return 0 in all non-missing components of the format, and the default for missing components.

Writes to non-mapped tiles are stopped from going to memory but might end up in caches that subsequent reads to the same address might or might not pick up.

Texture filtering

Texture filtering with a footprint that straddles **NULL** and non-**NULL** tiles contributes 0 (with defaults for missing format components) for texels on **NULL** tiles into the overall filter operation. Some early hardware don't meet this requirement and returns 0 (with defaults for missing format components) for the full filter result if any texels (with nonzero weight) fall on a **NULL** tile. No other hardware will be allowed to miss the requirement to include all (nonzero weighted) texels in the filter operation.

NULL texel accesses cause the [CheckAccessFullyMapped](#) operation on the status feedback for a texture read to return false. This is regardless of how the texture access

result might get write masked in the shader and how many components are in the texture format (the combination of which might make it appear that the texture does not need to be accessed).

Alignment constraints

Alignment constraints for standard tile shapes: Mipmaps that fill at least one standard tile in all dimensions are guaranteed to use the standard tiling, with the remainder considered packed as a **unit** into N tiles (N reported to the application). The application can map the N tiles into arbitrarily disjoint locations in a tile pool, but must either map all or none of the packed tiles. The mip packing is a unique set of packed tiles per array slice.

Min/Max reduction filtering

Min/Max reduction filtering is supported. See [Streaming resources texture sampling features](#).

Limitations

Streaming resources with any mipmaps less than standard tile size in any dimension are not allowed to have an array size larger than 1.

Limitations on how tiles can be accessed when there are duplicate mappings continue to apply. See [Tile access limitations with duplicate mappings](#).

Related topics

[Streaming resources features tiers](#)

Tier 3

Article • 10/20/2022

Tier 3 adds support for Texture3D for streaming resources, in addition to the [Tier 2](#) capabilities.

Related topics

[Streaming resources features tiers](#)

Appendices

Article • 10/20/2022

These sections provide in-depth technical details.

In this section

Topic	Description
Floating-point rules	Direct3D supports several floating-point representations. All floating-point computations operate under a defined subset of the IEEE 754 32-bit single precision floating-point rules.
Data type conversion	The following sections describe how Direct3D handles conversions between data types.
Rasterization rules	Rasterization rules define how vector data is mapped into raster data. The raster data is snapped to integer locations that are then culled and clipped (to draw the minimum number of pixels), and per-pixel attributes are interpolated (from per-vertex attributes) before being passed to a pixel shader.
Texture block compression	Block Compression (BC) support for textures has been extended in Direct3D 11 to include the BC6H and BC7 algorithms. BC6H supports high-dynamic range color source data, and BC7 provides better-than-average quality compression with less artifacts for standard RGB source data.

Related topics

[Direct3D Graphics Learning Guide](#)

Floating-point rules

Article • 10/20/2022

Direct3D supports several floating-point representations. All floating-point computations operate under a defined subset of the IEEE 754 32-bit single precision floating-point rules.

32-bit floating-point rules

There are two sets of rules: those that conform to IEEE-754, and those that deviate from the standard.

Honored IEEE-754 rules

Some of these rules are a single option where IEEE-754 offers choices.

- Divide by 0 produces +/- INF, except 0/0 which results in NaN.
- $\log$ of (+/-) 0 produces -INF.
- $\log$ of a negative value (other than -0) produces NaN.
- Reciprocal square root (rsq) or square root (sqrt) of a negative number produces NaN.

The exception is -0; sqrt(-0) produces -0, and rsq(-0) produces -INF.

- $\text{INF} - \text{INF} = \text{NaN}$
- $(+/-)\text{INF} / (+/-)\text{INF} = \text{NaN}$
- $(+/-)\text{INF} * 0 = \text{NaN}$
- $\text{NaN (any OP) any-value} = \text{NaN}$
- The comparisons EQ, GT, GE, LT, and LE, when either or both operands is NaN returns **FALSE**.
- Comparisons ignore the sign of 0 (so +0 equals -0).
- The comparison NE, when either or both operands is NaN returns **TRUE**.
- Comparisons of any non-NaN value against +/- INF return the correct result.

Deviations or additional requirements from IEEE-754 rules

- IEEE-754 requires floating-point operations to produce a result that is the nearest representable value to an infinitely-precise result, known as round-to-nearest-even.

Direct3D 11 and up define the same requirement as IEEE-754: 32-bit floating-point operations produce a result that is within 0.5 unit-last-place (ULP) of the infinitely-precise result. This means that, for example, hardware is allowed to truncate results to 32-bit rather than perform round-to-nearest-even, as that would result in error of at most 0.5 ULP. This rule applies only to addition, subtraction, and multiplication.

Earlier versions of Direct3D define a looser requirement than IEEE-754: 32-bit floating-point operations produce a result that is within one unit-last-place (1 ULP) of the infinitely-precise result. This means that, for example, hardware is allowed to truncate results to 32-bit rather than perform round-to-nearest-even, as that would result in error of at most one ULP.

- There is no support for floating-point exceptions, status bits or traps.
- Denorms are flushed to sign-preserved zero on input and output of any floating-point mathematical operation. Exceptions are made for any I/O or data movement operation that doesn't manipulate the data.
- States that contain floating-point values, such as Viewport MinDepth/MaxDepth or BorderColor values, may be provided as denorm values and may or may not be flushed before the hardware uses them.
- Min or max operations flush denorms for comparison, but the result may or may not be denorm flushed.
- NaN input to an operation always produces NaN on output. But the exact bit pattern of the NaN is not required to stay the same (unless the operation is a raw move instruction - which doesn't alter data.)
- Min or max operations for which only one operand is NaN return the other operand as the result (contrary to comparison rules we looked at earlier). This is an IEEE 754R rule.

The IEEE-754R specification for floating point min and max operations states that if one of the inputs to min or max is a quiet QNaN value, the result of the operation is the other parameter. For example:

ManagedCPlusPlus

```
min(x,QNaN) == min(QNaN,x) == x (same for max)
```

A revision of the IEEE-754R specification adopted a different behavior for min and max when one input is a "signaling" SNaN value versus a QNaN value:

ManagedCPlusPlus

```
min(x,SNaN) == min(SNaN,x) == QNaN (same for max)
```

Generally, Direct3D follows the standards for arithmetic: IEEE-754 and IEEE-754R. But in this case, we have a deviation.

The arithmetic rules in Direct3D 10 and later don't make any distinctions between quiet and signaling NaN values (QNaN versus SNaN). All NaN values are handled the same way. In the case of min and max, the Direct3D behavior for any NaN value is like how QNaN is handled in the IEEE-754R definition. (For completeness - if both inputs are NaN, any NaN value is returned.)

- Another IEEE 754R rule is that $\min(-0,+0) == \min(+0,-0) == -0$, and $\max(-0,+0) == \max(+0,-0) == +0$, which honors the sign, in contrast to the comparison rules for signed zero (as we saw earlier). Direct3D recommends the IEEE 754R behavior here, but doesn't enforce it; it is permissible for the result of comparing zeros to be dependent on the order of parameters, using a comparison that ignores the signs.
- $x * 1.0f$ always results in x (except denorm flushed).
- $x / 1.0f$ always results in x (except denorm flushed).
- $x +/- 0.0f$ always results in x (except denorm flushed). But $-0 + 0 = +0$.
- Fused operations (such as mad, dp3) produce results that are no less accurate than the worst possible serial ordering of evaluation of the unfused expansion of the operation. The definition of the worst possible ordering, for the purpose of tolerance, is not a fixed definition for a given fused operation; it depends on the particular values of the inputs. The individual steps in the unfused expansion are each allowed 1 ULP tolerance (or for any instructions Direct3D calls out with a more lax tolerance than 1 ULP, the more lax tolerance is allowed).
- Fused operations adhere to the same NaN rules as non-fused operations.

- sqrt and rcp have 1 ULP tolerance. The shader reciprocal and reciprocal square-root instructions, **rcp** and **rsq**, have their own separate relaxed precision requirement.
- Multiply and divide each operate at the 32-bit floating-point precision level (accuracy to 0.5 ULP for multiply, 1.0 ULP for reciprocal). If x/y is implemented directly, results must be of greater or equal accuracy than a two-step method.

64-bit (double precision) floating point rules

Hardware and display drivers optionally support double-precision floating-point. To indicate support, when you call **ID3D11Device::CheckFeatureSupport** with **D3D11_FEATURE_DOUBLES**, the driver sets **DoublePrecisionFloatShaderOps** of **D3D11_FEATURE_DATA_DOUBLES** to TRUE. The driver and hardware must then support all double-precision floating-point instructions.

Double-precision instructions follow IEEE 754R behavior requirements.

Support for generation of denormalized values is required for double-precision data (no flush-to-zero behavior). Likewise, instructions don't read denormalized data as a signed zero, they honor the denorm value.

16-bit floating-point rules

Direct3D also supports 16-bit representations of floating-point numbers.

Format:

- 1 sign bit (s) in the MSB bit position
- 5 bits of biased exponent (e)
- 10 bits of fraction (f), with an additional hidden bit

A float16 value (v) follows these rules:

- if $e == 31$ and $f \neq 0$, then v is NaN regardless of s
- if $e == 31$ and $f == 0$, then $v = (-1)^s * \text{infinity}$ (signed infinity)
- if e is between 0 and 31, then $v = (-1)^s * 2^{(e-15)} * (1.f)$
- if $e == 0$ and $f \neq 0$, then $v = (-1)^s * 2^{(e-14)} * (0.f)$ (denormalized numbers)
- if $e == 0$ and $f == 0$, then $v = (-1)^s * 0$ (signed zero)

32-bit floating-point rules also hold for 16-bit floating-point numbers, adjusted for the bit layout described earlier. Exceptions to this include:

- Precision: Unfused operations on 16-bit floating-point numbers produce a result that is the nearest representable value to an infinitely-precise result (round to nearest even, per IEEE-754, applied to 16-bit values). 32-bit floating-point rules adhere to 1 ULP tolerance, 16-bit floating-point rules adhere to 0.5 ULP for unfused operations, and 0.6 ULP for fused operations.
- 16-bit floating-point numbers preserve denorms.

11-bit and 10-bit floating-point rules

Direct3D also supports 11-bit and 10-bit floating-point formats.

Format:

- No sign bit
- 5 bits of biased exponent (e)
- 6 bits of fraction (f) for an 11-bit format, 5 bits of fraction (f) for a 10-bit format, with an additional hidden bit in either case.

A float11/float10 value (v) follows the following rules:

- if $e == 31$ and $f != 0$, then v is NaN
- if $e == 31$ and $f == 0$, then $v = +\text{infinity}$
- if e is between 0 and 31, then $v = 2^{(e-15)} \cdot (1.f)$
- if $e == 0$ and $f != 0$, then $v = 2^{(e-14)} \cdot (0.f)$ (denormalized numbers)
- if $e == 0$ and $f == 0$, then $v = 0$ (zero)

32-bit floating-point rules also hold for 11-bit and 10-bit floating-point numbers, adjusted for the bit layout described earlier. Exceptions include:

- Precision: 32-bit floating-point rules adhere to 0.5 ULP.
- 10/11-bit floating-point numbers preserve denorms.
- Any operation that would result in a number less than zero is clamped to zero.

Related topics

[Appendices](#)

[Resources](#)

[Textures](#)

Data type conversion

Article • 10/20/2022

The following sections describe how Direct3D handles conversions between data types.

Data Type Terminology

The following set of terms are subsequently used to characterize various format conversions.

 Expand table

Term	Definition
SNORM	Signed normalized integer, meaning that for an n-bit 2's complement number, the maximum value means 1.0f (for example, the 5-bit value 01111 maps to 1.0f), and the minimum value means -1.0f (for example, the 5-bit value 10000 maps to -1.0f). In addition, the second-minimum number maps to -1.0f (for example, the 5-bit value 10001 maps to -1.0f). There are thus two integer representations for -1.0f. There is a single representation for 0.0f, and a single representation for 1.0f. This results in a set of integer representations for evenly spaced floating point values in the range (-1.0f...0.0f), and also a complementary set of representations for numbers in the range (0.0f...1.0f)
UNORM	Unsigned normalized integer, meaning that for an n-bit number, all 0's means 0.0f, and all 1's means 1.0f. A sequence of evenly spaced floating point values from 0.0f to 1.0f are represented. for example, a 2-bit UNORM represents 0.0f, 1/3, 2/3, and 1.0f.
SINT	Signed integer. 2's complement integer. for example, an 3-bit SINT represents the integral values -4, -3, -2, -1, 0, 1, 2, 3.
UINT	Unsigned integer. for example, a 3-bit UINT represents the integral values 0, 1, 2, 3, 4, 5, 6, 7.
FLOAT	A floating-point value in any of the representations defined by Direct3D.
SRGB	Similar to UNORM, in that for an n-bit number, all 0's means 0.0f and all 1's means 1.0f. However unlike UNORM, with SRGB the sequence of unsigned integer encodings between all 0's to all 1's represent a nonlinear progression in the floating point interpretation of the numbers, between 0.0f to 1.0f. Roughly, if this nonlinear progression, SRGB, is displayed as a sequence of colors, it would appear as a linear ramp of luminosity levels to an "average" observer, under "average" viewing conditions, on an "average" display. For complete detail, refer to the SRGB color standard, IEC 61996-2-1, at IEC (International Electrotechnical Commission).

The terms above are often used as "Format Name Modifiers", where they describe both how data is laid out in memory and what conversion to perform in the transport path (potentially including filtering) from memory to or from a Pipeline unit such as a Shader.

Floating Point Conversion

Whenever a floating point conversion between different representations occurs, including to or from non-floating point representations, the following rules apply.

Converting from a higher range representation to a lower range representation

- Round-to-zero is used during conversion to another float format. If the target is an integer or fixed point format, round-to-nearest-even is used, unless the conversion is explicitly documented as using another rounding behavior, such as round-to-nearest for FLOAT to SNORM, FLOAT to UNORM or FLOAT to SRGB. Other exceptions are the ftoi and ftou shader instructions, which use round-to-zero. Finally, the float-to-fixed conversions used by the texture sampler and rasterizer have a specified tolerance measured in Unit-Last-Place from an infinitely precise ideal.

- For source values greater than the dynamic range of a lower range target format (eg. a large 32-bit float value is written into a 16-bit float RenderTarget), the maximum representable (appropriately signed) value results, NOT including signed infinity (due to the round to zero described above).
- NaN in a higher range format will be converted to NaN representation in the lower range format if the NaN representation exists in the lower range format. If the lower format does not have a NaN representation, the result will be 0.
- INF in a higher range format will be converted to INF in the lower range format if available. If the lower format does not have an INF representation, it will be converted to the maximum value representable. The sign will be preserved if available in the target format.
- Denorm in a higher range format will be converted to the Denorm representation in the lower range format if available in the lower range format and the conversion is possible, otherwise the result is 0. The sign bit will be preserved if available in the target format.

Converting from a lower range representation to a higher range representation

- NaN in a lower range format will be converted to the NaN representation in the higher range format if available in the higher range format. If the higher range format does not have a NaN representation, it will be converted to 0.
- INF in a lower range format will be converted to the INF representation in the higher range format if available in the higher range format. If the higher format does not have an INF representation, it will be converted to the maximum value representable (MAX_FLOAT in that format). The sign will be preserved if available in the target format.
- Denorm in a lower range format will be converted to a normalized representation in the higher range format if possible, or else to a Denorm representation in the higher range format if the Denorm representation exists. Failing those, if the higher range format does not have a Denorm representation, it will be converted to 0. The sign will be preserved if available in the target format. Note that 32-bit float numbers count as a format without a Denorm representation (because Denorms in operations on 32-bit floats flush to sign preserved 0).

Integer Conversion

The following table describes conversions from various representations described above to other representations. Only conversions that actually occur in Direct3D are shown.

With integers, unless otherwise specified, all conversions to/from integer representations to float representations described below will be done exactly.

[Expand table](#)

Source Data Type	Destination Data Type	Conversion Rule
SNORM	FLOAT	Given an n-bit integer value representing the signed range [-1.0f to 1.0f], conversion to floating-point is as follows. • The most-negative value maps to -1.0f. for example, the 5-bit value 10000 maps to -1.0f. • Every other value is converted to a float (call it c), and then result = c * (1.0f / (2⁽ⁿ⁻¹⁾-1)). For example the 5-bit value 10001 is converted to -15.0f, and then divided by 15.0f, yielding -1.0f.
FLOAT	SNORM	Given a floating-point number, conversion to an n-bit integer value representing the signed range [-1.0f to 1.0f] is as follows. • Let c represent the starting value. • If c is NaN, the result is 0. • If c > 1.0f, including INF, it is clamped to 1.0f. • If c < -1.0f, including -INF, it is clamped to -1.0f. • Convert from float scale to integer scale: c = c * (2⁽ⁿ⁻¹⁾-1). • Convert to an integer as follows. ◦ If c >= 0 then c = c + 0.5f, otherwise, c = c - 0.5f. ◦ Drop the decimal fraction, and the remaining floating point (integral) value is converted directly to an integer.

Source Data Type	Destination Data Type	Conversion Rule
		This conversion is permitted a tolerance of D3Dxx_FLOAT32_TO_INTEGER_TOLERANCE_IN_Unit-Last-Place Unit-Last-Place (on the integer side). This means that after converting from float to integer scale, any value within D3Dxx_FLOAT32_TO_INTEGER_TOLERANCE_IN_ULP Unit-Last-Place of a representable target format value is permitted to map to that value. The additional Data Invertability requirement ensures that the conversion is nondecreasing across the range and all output values are attainable. (In the constants shown here, xx should be replaced with the Direct3D version, for example 10, 11, or 12.)
UNORM	FLOAT	The starting n-bit value is converted to float (0.0f, 1.0f, 2.0f, etc.) and then divided by $(2^n - 1)$.
FLOAT	UNORM	Let c represent the starting value. • If c is NaN, the result is 0. • If $c > 1.0f$, including INF, it is clamped to 1.0f. • If $c < 0.0f$, including -INF, it is clamped to 0.0f. • Convert from float scale to integer scale: $c = c * (2^n - 1)$. • Convert to integer. ◦ $c = c + 0.5f$. ◦ The decimal fraction is dropped, and the remaining floating point (integral) value is converted directly to an integer. This conversion is permitted a tolerance of D3Dxx_FLOAT32_TO_INTEGER_TOLERANCE_IN_ULP Unit-Last-Place (on the integer side). This means that after converting from float to integer scale, any value within D3Dxx_FLOAT32_TO_INTEGER_TOLERANCE_IN_ULP Unit-Last-Place of a representable target format value is permitted to map to that value. The additional Data Invertability requirement ensures that the conversion is nondecreasing across the range and all output values are attainable.
SRGB	FLOAT	The following is the ideal SRGB to FLOAT conversion. • Take the starting n-bit value, convert it a float (0.0f, 1.0f, 2.0f, etc.); call this c. • $c = c * (1.0f / (2^n - 1))$ • If $(c \leq D3Dxx_SRGB_TO_FLOAT_THRESHOLD)$ then: $result = c / D3Dxx_SRGB_TO_FLOAT_DENOMINATOR_1$, else: $result = ((c + D3Dxx_SRGB_TO_FLOAT_OFFSET) / D3Dxx_SRGB_TO_FLOAT_DENOMINATOR_2) D3Dxx_SRGB_TO_FLOAT_EXPONENT$ This conversion is permitted a tolerance of D3Dxx_SRGB_TO_FLOAT_TOLERANCE_IN_ULP Unit-Last-Place (on the SRGB side).
FLOAT	SRGB	The following is the ideal FLOAT -> SRGB conversion. Assuming the target SRGB color component has n bits: • Suppose the starting value is c. • If c is NaN, the result is 0. • If $c > 1.0f$, including INF, is clamped to 1.0f. • If $c < 0.0f$, including -INF, it is clamped to 0.0f. • If $(c \leq D3Dxx_FLOAT_TO_SRGB_THRESHOLD)$ then: $c = D3Dxx_FLOAT_TO_SRGB_SCALE_1 * c$, else: $c = D3Dxx_FLOAT_TO_SRGB_SCALE_2 * c(D3Dxx_FLOAT_TO_SRGB_EXPONENT_NUMERATOR / D3Dxx_FLOAT_TO_SRGB_EXPONENT_DENOMINATOR) - D3Dxx_FLOAT_TO_SRGB_OFFSET$ • Convert from float scale to integer scale: $c = c * (2^n - 1)$. • Convert to integer: ◦ $c = c + 0.5f$. ◦ The decimal fraction is dropped, and the remaining floating point (integral) value is converted directly to an integer. This conversion is permitted a tolerance of D3Dxx_FLOAT32_TO_INTEGER_TOLERANCE_IN_ULP Unit-Last-Place (on the integer side). This means that after converting from float to integer scale, any value within D3Dxx_FLOAT32_TO_INTEGER_TOLERANCE_IN_ULP Unit-Last-Place of a representable target format value is permitted to map to that value. The additional Data Invertability requirement ensures that the conversion is nondecreasing across the range and all output values are attainable.
SINT	SINT With More Bits	To convert from SINT to an SINT with more bits, the most significant bit (MSB) of the starting number is "sign-extended" to the additional bits available in the target format.

Source Data Type	Destination Data Type	Conversion Rule
UINT	SINT With More Bits	To convert from UINT to an SINT with more bits, the number is copied to the target format's least significant bits (LSBs) and additional MSBs are padded with 0.
SINT	UINT With More Bits	To convert from SINT to UINT with more bits: If negative, the value is clamped to 0. Otherwise the number is copied to the target format's LSBs and additional MSB's are padded with 0.
UINT	UINT With More Bits	To convert from UINT to UINT with more bits the number is copied to the target format's LSBs and additional MSB's are padded with 0.
SINT or UINT	SINT or UINT With Fewer or Equal Bits	To convert from a SINT or UINT to SINT or UINT with fewer or equal bits (and/or change in signedness), the starting value is simply clamped to the range of the target format.

Fixed Point Integer Conversion

Fixed point integers are simply integers of some bit size that have an implicit decimal point at a fixed location.

The ubiquitous "integer" data type is a special case of a fixed point integer with the decimal at the end of the number.

Fixed point number representations are characterized as: i.f, where i is the number of integer bits and f is the number of fractional bits. for example, 16.8 means 16 bits integer followed by 8 bits of fraction. The integer part is stored in 2's complement, at least as defined here (though it can be defined equally for unsigned integers as well). The fractional part is stored in unsigned form. The fractional part always represents the positive fraction between the two nearest integral values, starting from the most negative.

Addition and subtraction operations on fixed point numbers are performed simply using standard integer arithmetic, without any consideration for where the implied decimal lies. Adding 1 to a 16.8 fixed point number just means adding 256, since the decimal is 8 places in from the least significant end of the number. Other operations such as multiplication, can be performed as well simply using integer arithmetic, provided the effect on the fixed decimal is accounted for. For example, multiplying two 16.8 integers using an integer multiply produces a 32.16 result.

Fixed point integer representations are used in two ways in Direct3D.

- Post-clipped vertex positions in the rasterizer are snapped to fixed point, to uniformly distribute precision across the RenderTarget area. Many rasterizer operations, including face culling as one example, occur on fixed point snapped positions, while other operations, such as attribute interpolator setup, use positions that have been converted back to floating point from the fixed point snapped positions.
- Texture coordinates for sampling operations are snapped to fixed point (after being scaled by texture size), to uniformly distribute precision across texture space, in choosing filter tap locations/weights. Weight values are converted back to floating point before actual filtering arithmetic is performed.

[Expand table](#)

Source Data Type	Destination Data Type	Conversion Rule
FLOAT	Fixed Point Integer	The following is the general procedure for converting a floating point number n to a fixed point integer i.f, where i is the number of (signed) integer bits and f is the number of fractional bits. • Compute $\text{FixedMin} = -2^{(i-1)}$ • Compute $\text{FixedMax} = 2^{(i-1)} - 2^{(-f)}$ • If n is a NaN, result = 0; if n is +Inf, result = $\text{FixedMax} * 2^f$; if n is -Inf, result = $\text{FixedMin} * 2^f$

Source Data Type	Destination Data Type	Conversion Rule
		- If $n \geq \text{FixedMax}$, result = $\text{FixedMax} * 2^f$; if $n \leq \text{FixedMin}$, result = $\text{FixedMin} * 2^f$ - Else compute $n * 2^f$ and convert to integer. Implementations are permitted D3Dxx_FLOAT32_TO_INTEGER_TOLERANCE_IN_ULP Unit-Last-Place tolerance in the integer result, instead of the infinitely precise value $n * 2^f$ after the last step above.
Fixed Point Integer	FLOAT	Assume that the specific fixed point representation being converted to float does not contain more than a total of 24 bits of information, no more than 23 bits of which is in the fractional component. Suppose a given fixed point number, fxp, is in i.f form (i bits integer, f bits fraction). The conversion to float is akin to the following pseudocode. float result = (float)(fxp >> f) + // extract integer ((float)(fxp & (2^f - 1)) / (2^f)); // extract fraction

Related topics

[Appendices](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) [Get help at Microsoft Q&A](#)

Rasterization rules

Article • 10/20/2022

Rasterization rules define how vector data is mapped into raster data. The raster data is snapped to integer locations that are then culled and clipped (to draw the minimum number of pixels), and per-pixel attributes are interpolated (from per-vertex attributes) before being passed to a pixel shader.

There are several types of rules, which depend on the type of primitive that is being mapped, as well as whether or not the data uses multisampling to reduce aliasing. The following illustrations demonstrate how the corner cases are handled.

Triangle Rasterization Rules (Without Multisampling)

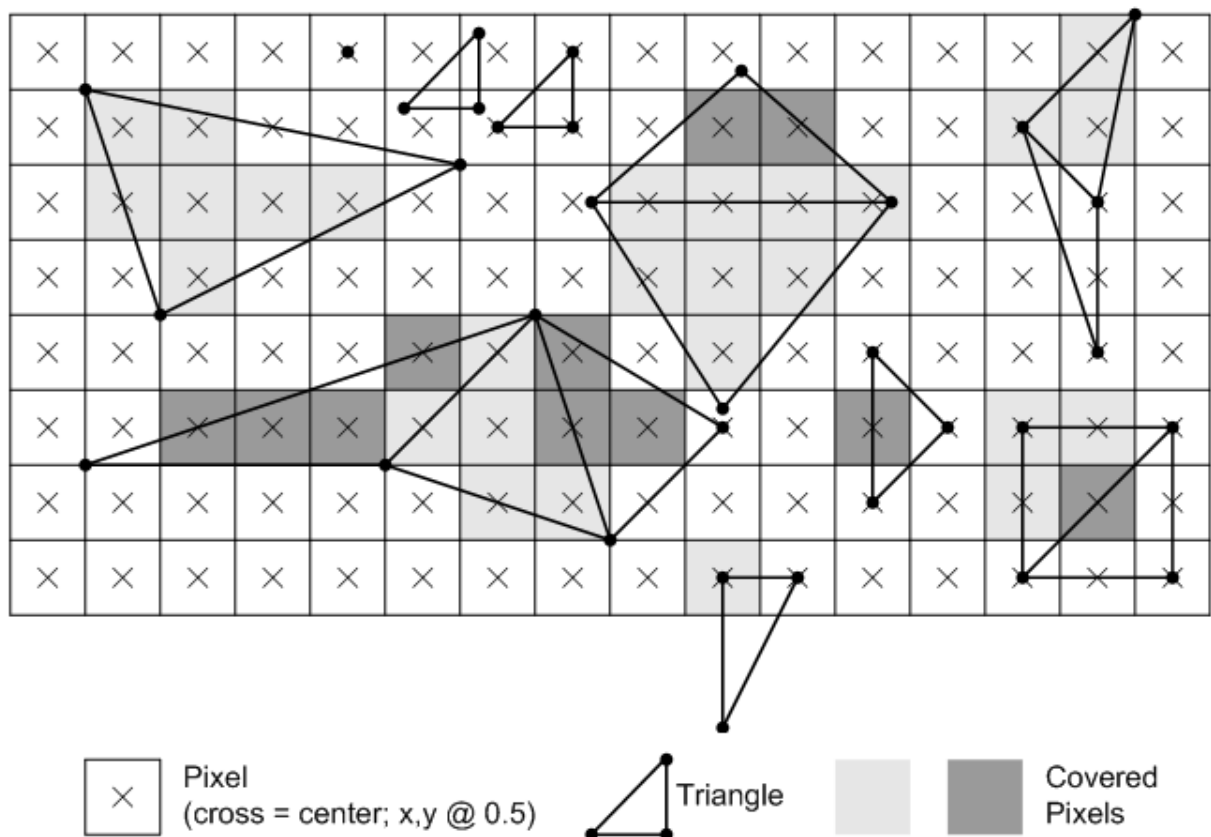
Any pixel center which falls inside a triangle is drawn; a pixel is assumed to be inside if it passes the top-left rule. The top-left rule is that a pixel center is defined to lie inside of a triangle if it lies on the top edge or the left edge of a triangle.

where:

- A top edge, is an edge that is exactly horizontal and is above the other edges.
- A left edge, is an edge that is not exactly horizontal and is on the left side of the triangle. A triangle can have one or two left edges.

The top-left rule ensures that adjacent triangles are drawn once.

This illustration shows examples of pixels that are drawn because they either lie inside a triangle or follow the top-left rule.



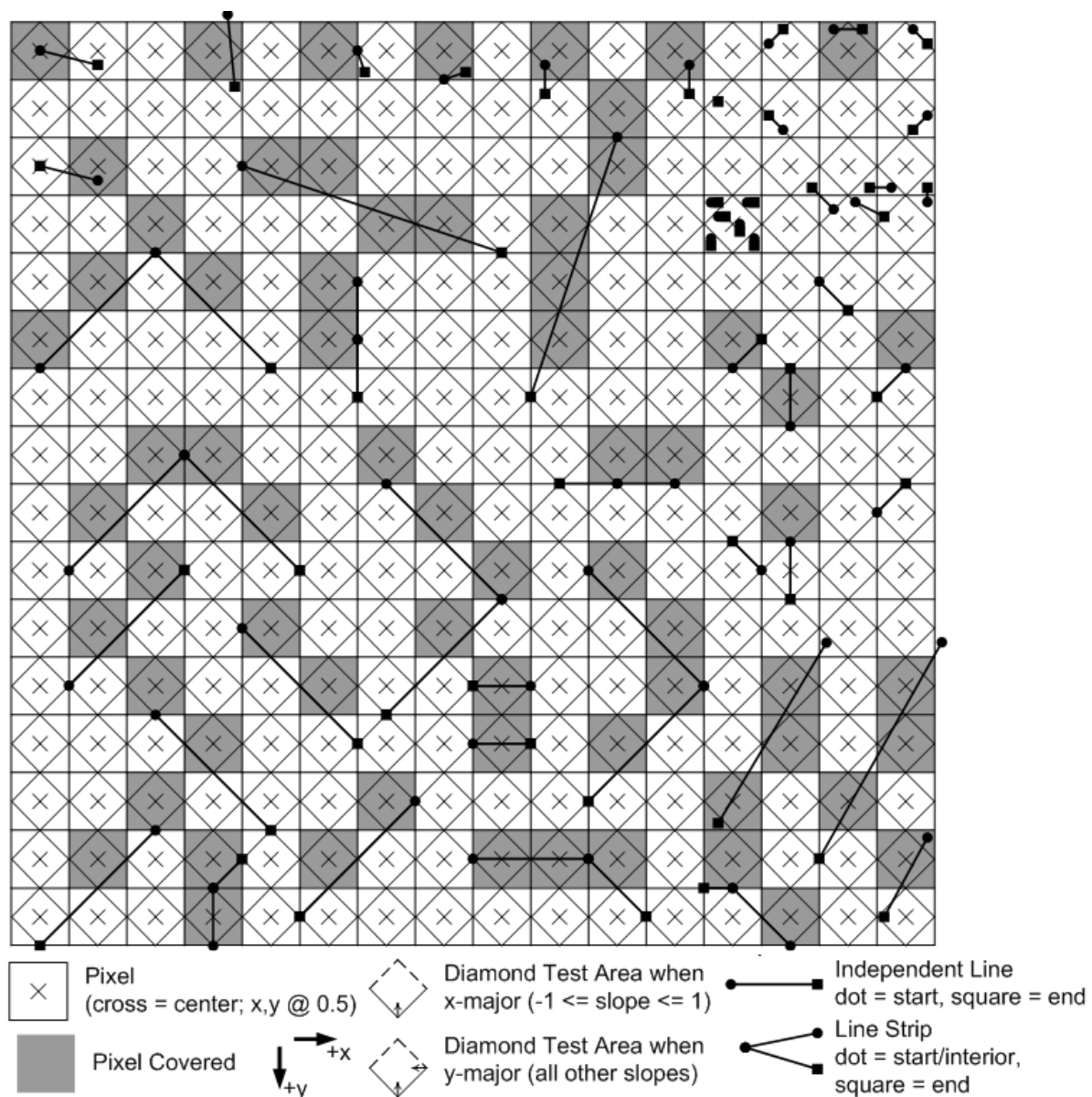
The light and dark gray covering of the pixels show them as groups of the pixels to indicate which triangle they are inside.

Line Rasterization Rules (Aliased, Without Multisampling)

Line rasterization rules use a diamond test area to determine if a line covers a pixel. For x-major lines (lines with $-1 \leq \text{slope} \leq +1$), the diamond test area includes (shown solid) the lower-left edge, lower-right edge, and bottom corner; the diamond excludes (shown dotted) the upper-left edge, upper-right edge, the top corner, the left corner, and the right corner. A y-major line is any line that is not an x-major line; the test diamond area is the same as described for the x-major line except the right corner is also included.

Given the diamond area, a line covers a pixel if the line exits the pixel's diamond test area when traveling along the line from the start towards the end. A line strip behaves the same, as it is drawn as a sequence of lines.

The following illustration shows some examples.



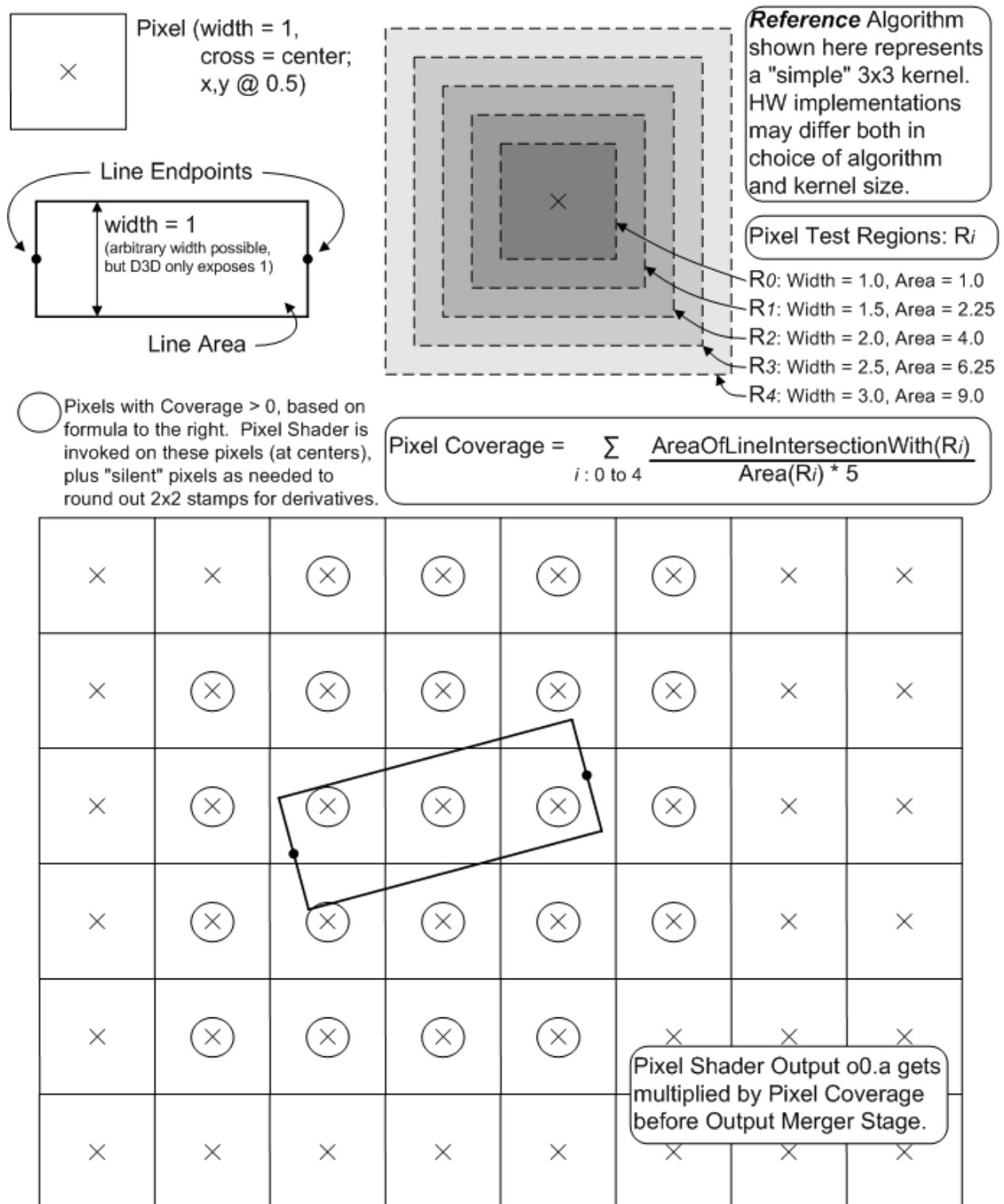
Line Rasterization Rules (Antialiased, Without Multisampling)

An antialiased line is rasterized as if it were a rectangle (with width = 1). The rectangle intersects with a render target producing per-pixel coverage values, which are multiplied into pixel shader output alpha components. There is no antialiasing preformed when drawing lines on a multisampled render target.

It is deemed that there is no single "best" way to perform antialiased line rendering. Direct3D adopts as a guideline the method shown in the following illustration. This method was derived empirically, exhibiting a number of visual properties deemed desirable.

Hardware need not exactly match this algorithm; tests against this reference shall have "reasonable" tolerances, guided by some of the principles listed further below,

permitting various hardware implementations and filter kernel sizes. None of this flexibility permitted in hardware implementation, however, can be communicated up through Direct3D to applications, beyond simply drawing lines and observing/measuring how they look.



This algorithm generates relatively smooth lines, with uniform intensity, with minimal jagged edges or braiding. Moire patterning for close lines is minimized. There is good coverage for junctions between line segments placed end-to-end.

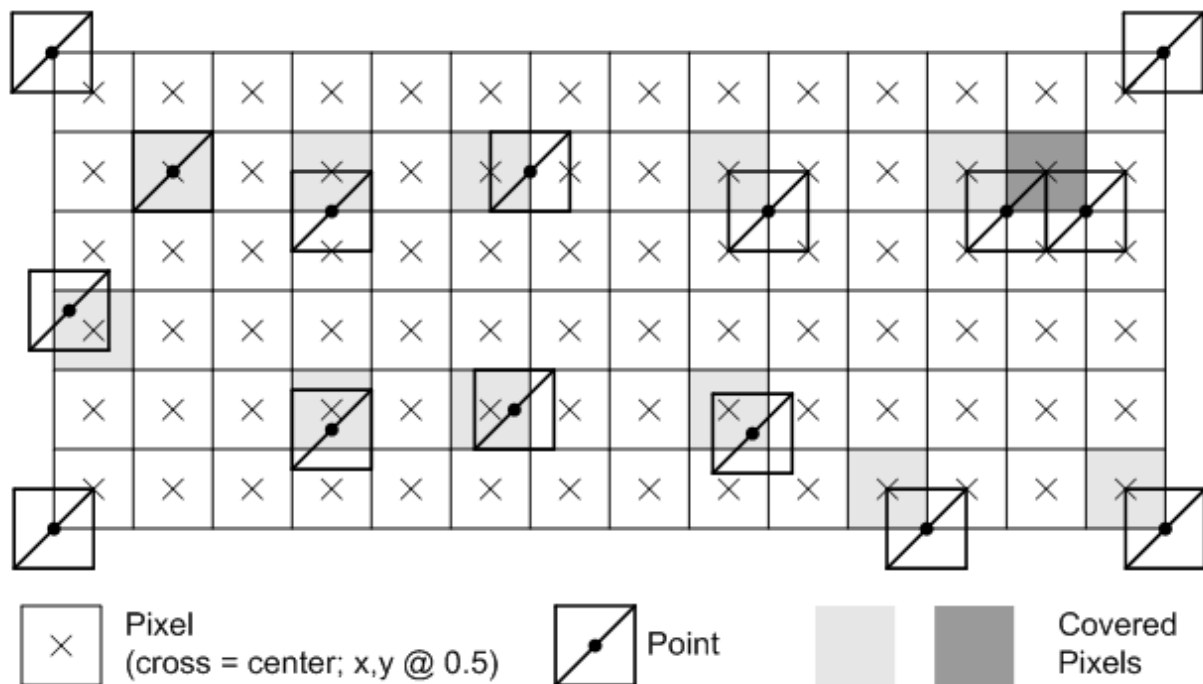
The filter kernel is a reasonable tradeoff between the amount of edge blurring and the changes in intensity caused by gamma corrections. The coverage value is multiplied into pixel shader o0.a (srcAlpha) per the following formula by the [Output Merger \(OM\)](#) stage:

$$\text{srcColor} * \text{srcAlpha} + \text{destColor} * (1 - \text{srcAlpha})$$

Point Rasterization Rules (Without Multisampling)

A point is interpreted as though it were composed of two triangles in a Z pattern, which use triangle rasterization rules. The coordinate identifies the center of a one pixel wide square. There is no culling for points.

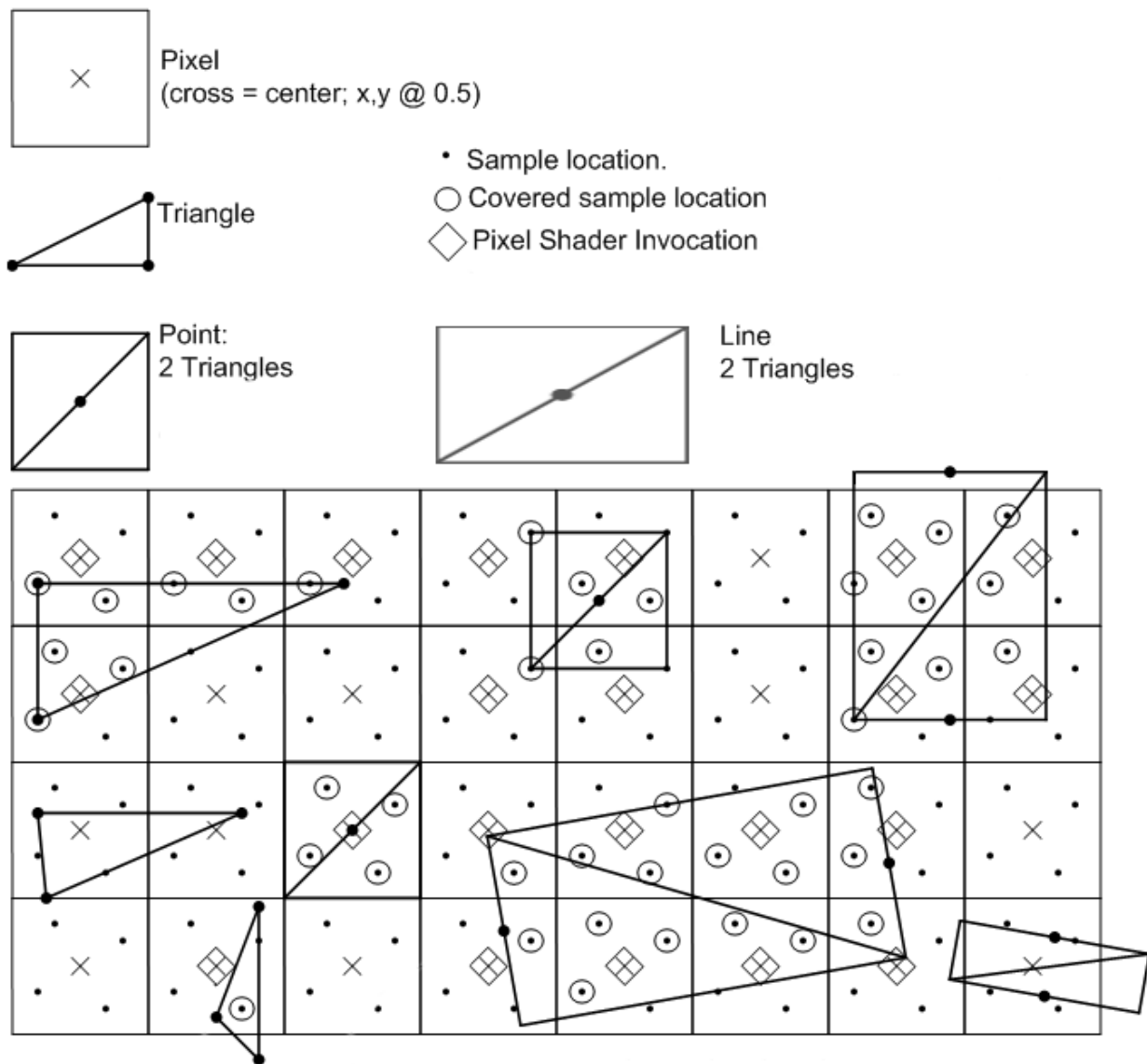
The following illustration shows some examples.



Multisample Anti-Aliasing Rasterization Rules

Multisample antialiasing (MSAA) reduces geometry aliasing using pixel coverage and depth-stencil tests at multiple sub-sample locations. To improve performance, per-pixel calculations are performed once for each covered pixel, by sharing shader outputs across covered sub-pixels. Multisample antialiasing does not reduce surface aliasing. Sample locations and reconstruction functions are dependent on the hardware implementation.

The following illustration shows some examples.



The number of sample locations is dependent on the multisample mode. Vertex attributes are interpolated at pixel centers, since this is where the pixel shader is invoked (this becomes extrapolation if the center is not covered). Attributes can be flagged in the pixel shader to be centroid sampled, which causes non-covered pixels to interpolate the attribute at intersection of the pixel's area and the primitive.

A pixel shader runs for each 2x2 pixel area to support derivative calculations (which use x and y deltas). This means that shader invocations occur more than is shown to fill out the minimum 2x2 quanta (which is independent of multisampling). The shader result is written out for each covered sample that passes the per-sample depth-stencil test.

Rasterization rules for primitives are, in general, unchanged by multisample antialiasing, except:

- For a triangle, a coverage test is performed for each sample location (not for a pixel center). If more than one sample location is covered, a pixel shader runs once with attributes interpolated at the pixel center. The result is stored (replicated) for each covered sample location in the pixel that passes the depth/stencil test.

A line is treated as a rectangle made up of two triangles, with a line width of 1.4.

- For a point, a coverage test is performed for each sample location (not for a pixel center).

Multisampling formats can be used in render targets which can be read back into shaders using [load](#), since no resolve is required for individual samples accessed by the shader. Depth formats are not supported for multisample resource, therefore, depth formats are restricted to render targets only.

Typeless formats support multisampling to allow a resource view to interpret data in different ways. For instance, you could create a multisample resource using `R8G8B8A8_TYPELESS`, render to it using a render-target-view resource with a `R8G8B8A8_UINT` format, then resolve the contents to another resource with a `R8G8B8A8_UNORM` data format.

Hardware Support

The API reports hardware support for multisampling through the number of quality levels. For example, a 0 quality level means the hardware does not support multisampling (at a particular format and quality level). A 3 for quality levels means that the hardware supports three different sample layouts and/or resolve algorithms. You can also assume the following:

- Any format that supports multisampling, supports the same number of quality levels for every format in that family.
- Every format that supports multisampling, and has the `_UNORM`, `_SRGB`, `_SNORM` or `_FLOAT` formats, also supports resolving.

Centroid Sampling of Attributes when Multisample Antialiasing

By default, vertex attributes are interpolated to a pixel center during multisample antialiasing; if the pixel center is not covered, attributes are extrapolated to a pixel center. If a pixel shader input that contains the centroid semantic (assuming the pixel is not fully covered) will be sampled somewhere within the covered area of the pixel, possibly at one of the covered sample locations. A sample mask (specified by the rasterizer state) is applied prior to centroid computation. Therefore, a sample that is masked out will not be used as a centroid location.

The reference rasterizer chooses a sample location for centroid sampling similar to this:

- The sample mask allows all samples. Use a pixel center if the pixel is covered or if none of the samples are covered. Otherwise, the first covered sample is chosen, starting from the pixel center and moving outward.
- The sample mask turns off all samples but one (a common scenario). An application can implement multipass supersampling by cycling through single-bit sample-mask values and re-rendering the scene for each sample using centroid sampling. This would require that an application adjust derivatives to select appropriately more detailed texture mips for the higher texture sampling density.

Derivative Calculations When Multisampling

Pixel shaders always run using a minimum 2x2 pixel area to support derivative calculations, which are calculated by taking deltas between data from adjacent pixels (making the assumption that the data in each pixel has been sampled with unit spacing horizontally or vertically). This is unaffected by multisampling.

If derivatives are requested on an attribute that has been centroid sampled, the hardware calculation is not adjusted, which can cause inaccurate derivatives. A shader will expect a unit vector in render-target space but may get a non-unit vector with respect to some other vector space. Therefore, it is an application's responsibility to exhibit caution when requesting derivatives from attributes that are centroid sampled.

In fact, it is recommended that you do not combine derivatives and centroid sampling. Centroid sampling can be useful for situations where it is critical that a primitive's interpolated attributes are not extrapolated, but this comes with tradeoffs such as attributes that appear to jump where a primitive edge crosses a pixel (rather than change continuously) or derivatives that cannot be used by texture sampling operations that derive LOD.

Related topics

[Appendices](#)

[Rasterizer \(RS\) stage](#)

Texture block compression

Article • 10/20/2022

Block Compression (BC) support for textures has been extended in Direct3D 11 to include the BC6H and BC7 algorithms. BC6H supports high-dynamic range color source data, and BC7 provides better-than-average quality compression with less artifacts for standard RGB source data.

For more specific information about block compression algorithm support prior to Direct3D 11, including support for the BC1 through BC5 formats, see [Block Compression \(Direct3D 10\)](#).

****Note about File Formats:** ** The BC6H and BC7 texture compression formats use the DDS file format for storing the compressed texture data. For more information, see the [Programming Guide for DDS](#) for details.

Block Compression Formats Supported in Direct3D 11

Source data	Minimum required data compression resolution	Recommended format	Minimum supported feature level
Three-channel color with alpha channel	Three color channels (5 bits:6 bits:5 bits), with 0 or 1 bit(s) of alpha	BC1	Direct3D 9.1
Three-channel color with alpha channel	Three color channels (5 bits:6 bits:5 bits), with 4 bits of alpha	BC2	Direct3D 9.1
Three-channel color with alpha channel	Three color channels (5 bits:6 bits:5 bits) with 8 bits of alpha	BC3	Direct3D 9.1
One-channel color	One color channel (8 bits)	BC4	Direct3D 10
Two-channel color	Two color channels (8 bits:8 bits)	BC5	Direct3D 10
Three-channel high dynamic range (HDR) color	Three color channels (16 bits:16 bits:16 bits) in "half" floating point*	BC6H	Direct3D 11
Three-channel color, alpha channel optional	Three color channels (4 to 7 bits per channel) with 0 to 8 bits of alpha	BC7	Direct3D 11

*"Half" floating point is a 16 bit value that consists of an optional sign bit, a 5 bit biased exponent, and a 10 or 11 bit mantissa.

BC1, BC2, and B3 Formats

The BC1, BC2, and BC3 formats are equivalent to the Direct3D 9 DXTn texture compression formats, and are the same as the corresponding Direct3D 10 BC1, BC2, and BC3 formats. Support for these three formats is required by all feature levels (D3D_FEATURE_LEVEL_9_1, D3D_FEATURE_LEVEL_9_2, D3D_FEATURE_LEVEL_9_3, D3D_FEATURE_LEVEL_10_0, D3D_FEATURE_LEVEL_10_1, and D3D_FEATURE_LEVEL_11_0).

Block compression format	DXGI format	Direct3D 9 equivalent format	Bytes per 4x4 pixel block
BC1	DXGI_FORMAT_BC1_UNORM, DXGI_FORMAT_BC1_UNORM_SRGB, DXGI_FORMAT_BC1_TYPELESS	D3DFMT_DXT1, FourCC="DXT1"	8
BC2	DXGI_FORMAT_BC2_UNORM, DXGI_FORMAT_BC2_UNORM_SRGB, DXGI_FORMAT_BC2_TYPELESS	D3DFMT_DXT2*, FourCC="DXT2", D3DFMT_DXT3, FourCC="DXT3"	16
BC3	DXGI_FORMAT_BC3_UNORM, DXGI_FORMAT_BC3_UNORM_SRGB, DXGI_FORMAT_BC3_TYPELESS	D3DFMT_DXT4*, FourCC="DXT4", D3DFMT_DXT5, FourCC="DXT5"	16

*These compression schemes (DXT2 and DXT4) make no distinction between the Direct3D 9 pre-multiplied alpha formats and the standard alpha formats. This distinction must be handled by the programmable shaders at render time.

BC4 and BC5 Formats

Block compression format	DXGI format	Direct3D 9 equivalent format	Bytes per 4x4 pixel block
BC4	DXGI_FORMAT_BC4_UNORM, DXGI_FORMAT_BC4_SNORM, DXGI_FORMAT_BC4_TYPELESS	FourCC="ATI1"	8

Block compression format	DXGI format	Direct3D 9 equivalent format	Bytes per 4x4 pixel block
BC5	DXGI_FORMAT_BC5_UNORM, DXGI_FORMAT_BC5_SNORM, DXGI_FORMAT_BC5_TYPELESS	FourCC="ATI2"	16

BC6H Format

For more detailed information about this format, see the [BC6H Format](#) documentation.

Block compression format	DXGI format	Direct3D 9 equivalent format	Bytes per 4x4 pixel block
BC6H	DXGI_FORMAT_BC6H_UF16, DXGI_FORMAT_BC6H_SF16, DXGI_FORMAT_BC6H_TYPELESS	N/A	16

The BC6H format can select different encoding modes for each 4x4 pixel block. A total of 14 different encoding modes are available, each with slightly different trade-offs in the resulting visual quality of the displayed texture. The choice of modes allows for fast decoding by the hardware with the quality level selected or adapted according to the source content, but it also greatly increases the complexity of the search space.

BC7 Format

For more detailed information about this format, see the [BC7 Format](#) documentation.

Block compression format	DXGI format	Direct3D 9 equivalent format	Bytes per 4x4 pixel block
BC7	DXGI_FORMAT_BC7_UNORM, DXGI_FORMAT_BC7_UNORM_SRGB, DXGI_FORMAT_BC7_TYPELESS	N/A	16

The BC7 format can select different encoding modes for each 4x4 pixel block. A total of 8 different encoding modes are available, each with slightly different trade-offs in the resulting visual quality of the displayed texture. The choice of modes allows for fast decoding by the hardware with the quality level selected or adapted according to the source content, but it also greatly increases the complexity of the search space.

Related topics

[Appendices](#)

[Textures](#)

BC6H format

Article • 10/20/2022

The BC6H format is a texture compression format designed to support high-dynamic range (HDR) color spaces in source data.

About BC6H/DXGI_FORMAT_BC6H

The BC6H format provides high-quality compression for images that use three HDR color channels, with a 16-bit value for each color channel of the value (16:16:16). There is no support for an alpha channel.

BC6H is specified by the following DXGI_FORMAT enumeration values:

- **DXGI_FORMAT_BC6H_TYPELESS.**
- **DXGI_FORMAT_BC6H_UF16.** This BC6H format does not use a sign bit in the 16-bit floating point color channel values.
- **DXGI_FORMAT_BC6H_SF16.** This BC6H format uses a sign bit in the 16-bit floating point color channel values.

Note The 16 bit floating point format for color channels is often referred to as a "half" floating point format. This format has the following bit layout:

Format	Layout
UF16 (unsigned float)	5 exponent bits + 11 mantissa bits
SF16 (signed float)	1 sign bit + 5 exponent bits + 10 mantissa bits

The BC6H format can be used for [Texture2D](#) (including arrays), Texture3D, or TextureCube (including arrays) texture resources. Similarly, this format applies to any MIP-map surfaces associated with these resources.

BC6H uses a fixed block size of 16 bytes (128 bits) and a fixed tile size of 4x4 texels. As with previous BC formats, texture images larger than the supported tile size (4x4) are compressed by using multiple blocks. This addressing identity applies also to three-dimensional images, MIP-maps, cube maps, and texture arrays. All image tiles must be of the same format.

Caveats with the BC6H format:

- BC6H supports floating point denormalization, but does not support INF (infinity) and NaN (not a number). The exception is the signed mode of BC6H (DXGI_FORMAT_BC6H_SF16), which supports -INF (negative infinity). This support for -INF is merely an artifact of the format itself, and is not specifically supported by encoders for this format. In general, when an encoder encounters INF (positive or negative) or NaN input data, the encoder should convert that data to the maximum allowable non-INF representation value, and map NaN to 0 prior to compression.
- BC6H does not support an alpha channel.
- The BC6H decoder performs decompression before it performs texture filtering.
- BC6H decompression must be bit-accurate; that is, the hardware must return results that are identical to the decoder described in this documentation.

BC6H implementation

A BC6H block consists of mode bits, compressed endpoints, compressed indices, and an optional partition index. This format specifies 14 different modes.

An endpoint color is stored as an RGB triplet. BC6H defines a palette of colors on an approximate line across a number of defined color endpoints. Also, depending on the mode, a tile can be divided into two regions or treated as a single region, where a two-region tile has a separate set of color endpoints for each region. BC6H stores one palette index per texel.

In the two-region case, there are 32 possible partitions.

Decoding the BC6H format

The pseudocode below shows the steps to decompress the pixel at (x,y) given the 16 byte BC6H block.

syntax

```
decompress_bc6h(x, y, block)
{
    mode = extract_mode(block);
    endpoints;
    index;

    if(mode.type == ONE)
    {
        endpoints = extract_compressed_endpoints(mode, block);
```

```

        index = extract_index_ONE(x, y, block);
    }
    else //mode.type == TWO
    {
        partition = extract_partition(block);
        region = get_region(partition, x, y);
        endpoints = extract_compressed_endpoints(mode, region, block);
        index = extract_index_TWO(x, y, partition, block);
    }

    unquantize(endpoints);
    color = interpolate(index, endpoints);
    finish_unquantize(color);
}

```

The following table contains the bit count and values for each of the 14 possible formats for BC6H blocks.

Mode	Partition Indices	Partition	Color Endpoints	Mode Bits
1	46 bits	5 bits	75 bits (10.555, 10.555, 10.555)	2 bits (00)
2	46 bits	5 bits	75 bits (7666, 7666, 7666)	2 bits (01)
3	46 bits	5 bits	72 bits (11.555, 11.444, 11.444)	5 bits (00010)
4	46 bits	5 bits	72 bits (11.444, 11.555, 11.444)	5 bits (00110)
5	46 bits	5 bits	72 bits (11.444, 11.444, 11.555)	5 bits (01010)
6	46 bits	5 bits	72 bits (9555, 9555, 9555)	5 bits (01110)
7	46 bits	5 bits	72 bits (8666, 8555, 8555)	5 bits (10010)
8	46 bits	5 bits	72 bits (8555, 8666, 8555)	5 bits (10110)
9	46 bits	5 bits	72 bits (8555, 8555, 8666)	5 bits (11010)
10	46 bits	5 bits	72 bits (6666, 6666, 6666)	5 bits (11110)
11	63 bits	0 bits	60 bits (10.10, 10.10, 10.10)	5 bits (00011)
12	63 bits	0 bits	60 bits (11.9, 11.9, 11.9)	5 bits (00111)
13	63 bits	0 bits	60 bits (12.8, 12.8, 12.8)	5 bits (01011)
14	63 bits	0 bits	60 bits (16.4, 16.4, 16.4)	5 bits (01111)

Each format in this table can be uniquely identified by the mode bits. The first ten modes are used for two-region tiles, and the mode bit field can be either two or five bits

long. These blocks also have fields for the compressed color endpoints (72 or 75 bits), the partition (5 bits), and the partition indices (46 bits).

For the compressed color endpoints, the values in the preceding table note the precision of the stored RGB endpoints, and the number of bits used for each color value. For example, mode 3 specifies a color endpoint precision level of 11, and the number of bits used to store the delta values of the transformed endpoints for the red, blue and green colors (5, 4, and 4 respectively). Mode 10 does not use delta compression, and instead stores all four color endpoints explicitly.

The last four block modes are used for one-region tiles, where the mode field is 5 bits. These blocks have fields for the endpoints (60 bits) and the compressed indices (63 bits). Mode 11 (like Mode 10) does not use delta compression, and instead stores both color endpoints explicitly.

Modes 10011, 10111, 11011, and 11111 (not shown) are reserved. Do not use these in your encoder. If the hardware is passed blocks with one of these modes specified, the resulting decompressed block must contain all zeroes in all channels except for the alpha channel.

For BC6H, the alpha channel must always return 1.0 regardless of the mode.

BC6H partition set

There are 32 possible partition sets for a two-region tile, and which are defined in the table below. Each 4x4 block represents a single shape.

0, 0, <u>1</u> , 1,	0, 0, 0, <u>1</u> ,	0, <u>1</u> , 1, 1,	0, 0, 0, <u>1</u> ,
0, 0, <u>1</u> , 1,	0, 0, 0, <u>1</u> ,	0, <u>1</u> , 1, 1,	0, 0, <u>1</u> , 1,
0, 0, <u>1</u> , 1,	0, 0, 0, <u>1</u> ,	0, <u>1</u> , 1, 1,	0, 0, <u>1</u> , 1,
0, 0, <u>1</u> , <u>1</u> ,	0, 0, 0, <u>1</u> ,	0, <u>1</u> , 1, <u>1</u> ,	0, <u>1</u> , 1, <u>1</u> ,
0, 0, 0, 0,	0, 0, <u>1</u> , 1,	0, 0, 0, <u>1</u> ,	0, 0, 0, 0,
0, 0, 0, <u>1</u> ,	0, <u>1</u> , 1, 1,	0, 0, <u>1</u> , 1,	0, 0, 0, <u>1</u> ,
0, 0, 0, <u>1</u> ,	0, <u>1</u> , 1, 1,	0, <u>1</u> , 1, 1,	0, 0, <u>1</u> , 1,
0, 0, <u>1</u> , <u>1</u> ,	<u>1</u> , 1, 1, <u>1</u> ,	<u>1</u> , 1, 1, <u>1</u> ,	0, <u>1</u> , 1, <u>1</u> ,
0, 0, 0, 0,	0, 0, <u>1</u> , 1,	0, 0, 0, 0,	0, 0, 0, 0,
0, 0, 0, 0,	0, <u>1</u> , 1, 1,	0, 0, 0, <u>1</u> ,	0, 0, 0, 0,
0, 0, 0, <u>1</u> ,	<u>1</u> , 1, 1, 1,	0, <u>1</u> , 1, 1,	0, 0, 0, <u>1</u> ,
0, 0, <u>1</u> , <u>1</u> ,	<u>1</u> , 1, 1, <u>1</u> ,	<u>1</u> , 1, 1, <u>1</u> ,	0, <u>1</u> , 1, <u>1</u> ,
0, 0, 0, <u>1</u> ,	0, 0, 0, 0,	0, 0, 0, 0,	0, 0, 0, 0,
0, <u>1</u> , 1, 1,	0, 0, 0, 0,	<u>1</u> , 1, 1, 1,	0, 0, 0, 0,
<u>1</u> , 1, 1, 1,	<u>1</u> , 1, 1, 1,	<u>1</u> , 1, 1, 1,	0, 0, 0, 0,
<u>1</u> , 1, 1, <u>1</u> ,	<u>1</u> , 1, 1, <u>1</u> ,	<u>1</u> , 1, 1, <u>1</u> ,	<u>1</u> , 1, 1, <u>1</u> ,
0, 0, 0, 0,	0, <u>1</u> , <u>1</u> , 1,	0, 0, 0, 0,	0, <u>1</u> , <u>1</u> , 1,
<u>1</u> , 0, 0, 0,	0, 0, 0, <u>1</u> ,	0, 0, 0, 0,	0, 0, <u>1</u> , 1,
<u>1</u> , 1, <u>1</u> , 0,	0, 0, 0, 0,	<u>1</u> , 0, 0, 0,	0, 0, 0, <u>1</u> ,
<u>1</u> , 1, <u>1</u> , <u>1</u> ,	0, 0, 0, 0,	<u>1</u> , 1, 1, 0,	0, 0, 0, 0,
0, 0, <u>1</u> , 1,	0, 0, 0, 0,	0, 0, 0, 0,	0, <u>1</u> , 1, 1,
0, 0, 0, <u>1</u> ,	<u>1</u> , 0, 0, 0,	0, 0, 0, 0,	0, 0, <u>1</u> , 1,
0, 0, 0, 0,	<u>1</u> , 1, 0, 0,	<u>1</u> , 0, 0, 0,	0, 0, <u>1</u> , 1,
0, 0, 0, 0,	<u>1</u> , 1, 1, 0,	<u>1</u> , 1, 0, 0,	0, 0, 0, <u>1</u> ,
0, 0, <u>1</u> , 1,	0, 0, 0, 0,	0, <u>1</u> , <u>1</u> , 0,	0, 0, <u>1</u> , 1,
0, 0, 0, <u>1</u> ,	<u>1</u> , 0, 0, 0,	0, <u>1</u> , <u>1</u> , 0,	0, <u>1</u> , <u>1</u> , 0,
0, 0, 0, <u>1</u> ,	<u>1</u> , 0, 0, 0,	0, <u>1</u> , 1, 0,	0, <u>1</u> , 1, 0,
0, 0, 0, 0,	<u>1</u> , 1, 0, 0,	0, <u>1</u> , 1, 0,	<u>1</u> , 1, 0, 0,
0, 0, 0, <u>1</u> ,	0, 0, 0, 0,	0, <u>1</u> , <u>1</u> , 1,	0, 0, <u>1</u> , 1,
0, <u>1</u> , 1, 1,	<u>1</u> , 1, 1, 1,	0, 0, 0, <u>1</u> ,	<u>1</u> , 0, 0, <u>1</u> ,
<u>1</u> , 1, 1, 0,	<u>1</u> , 1, 1, 1,	<u>1</u> , 0, 0, 0,	<u>1</u> , 0, 0, <u>1</u> ,
<u>1</u> , 0, 0, 0,	0, 0, 0, 0,	<u>1</u> , 1, 1, 0,	<u>1</u> , 1, 0, 0,

In this table of partition sets, the bolded and underlined entry is the location of the fix-up index for subset 1 (which is specified with one less bit). The fix-up index for subset 0 is always index 0, as the partitioning is always arranged such that index 0 is always in subset 0. Partition order goes from top-left to bottom-right, moving left to right and then top to bottom.

BC6H compressed endpoint format

Header Bit	10 5 5 5	7 6 6 6	11 5 4 4	11 4 5 4	11 4 4 5	9 5 5 5	8 6 5 5	8 5 6 5	8 5 5 6	6 6 6 6	10 10	11 9	12 8	16 4
0														
1	m[1:0]	m[1:0]												
2	gy[4]	gy[5]												
3	by[4]	gz[4]												
4	bz[4]	gz[5]	m[4:0]	m[4:0]	m[4:0]	m[4:0]	m[4:0]	m[4:0]	m[4:0]	m[4:0]	m[4:0]	m[4:0]	m[4:0]	m[4:0]
5														
6														
7														
8														
9														
10														
11		rw[6:0]								rw[5:0]				
12		bz[0]					rw[7:0]	rw[7:0]	rw[7:0]	gz[4]				
13		bz[1]				rw[8:0]	gz[4]	bz[0]	bz[1]	bz[0]				
14	rw[9:0]	by[4]	rw[9:0]	rw[9:0]	rw[9:0]	by[4]	by[4]	by[4]	by[4]	by[4]	rw[9:0]	rw[9:0]	rw[9:0]	rw[9:0]
15														
16														
17														
18														
19														
20														
21		gw[6:0]								gw[5:0]				
22		by[5]					gw[7:0]	gw[7:0]	gw[7:0]	gy[5]				
23		bz[2]				gw[8:0]	bz[2]	gy[5]	by[5]	bz[2]				
24	gw[9:0]	gy[4]	gw[9:0]	gw[9:0]	gw[9:0]	gy[4]	gy[4]	gy[4]	gy[4]	gy[4]	gw[9:0]	gw[9:0]	gw[9:0]	gw[9:0]
25														
26														
27														
28														
29														
30														
31		bw[6:0]								bw[5:0]				
32		bz[3]					bw[7:0]	bw[7:0]	bw[7:0]	gz[5]				
33		bz[5]				bw[8:0]	bz[3]	gz[5]	bz[5]	bz[3]				
34	bw[9:0]	bz[4]	bw[9:0]	bw[9:0]	bw[9:0]	bz[4]	bz[4]	bz[4]	bz[4]	bz[4]	bw[9:0]	bw[9:0]	bw[9:0]	bw[9:0]
35														
36														
37														
38				rx[3:0]	rx[3:0]									rx[3:0]
39	rx[4:0]		rx[4:0]	rw[10]	rw[10]	rx[4:0]		rx[4:0]	rx[4:0]					
40	gz[4]	rx[5:0]	rw[10]	gz[4]	by[4]	gz[4]	rx[5:0]	gz[4]	gz[4]	rx[5:0]				
41														
42													rx[7:0]	
43														
44	gy[3:0]	gy[3:0]	gy[3:0]	gy[3:0]	gy[3:0]	gy[3:0]	gy[3:0]	gy[3:0]	gy[3:0]	gy[3:0]	rx[9:0]	rx[8:0]	rx[10:11]	rx[10:15]
45												rw[10]		
46														
47														
48			gx[3:0]		gx[3:0]									gx[3:0]
49	gx[4:0]		gw[10]	gx[4:0]	gw[10]	gx[4:0]	gx[4:0]		gx[4:0]					
50	bz[0]	gx[5:0]	bz[0]	gw[10]	bz[0]	bz[0]	bz[0]	gx[5:0]	bz[0]	gx[5:0]				
51														
52														
53														
54	gz[3:0]	gz[3:0]	gz[3:0]	gz[3:0]	gz[3:0]	gz[3:0]	gz[3:0]	gz[3:0]	gz[3:0]	gz[3:0]	gx[9:0]	gx[8:0]	gw[10:11]	gw[10:15]
55												gw[10]		
56														
57														
58			bx[3:0]	bx[3:0]										bx[3:0]
59	bx[4:0]		bw[10]	bx[4:0]	bx[4:0]	bx[4:0]	bx[4:0]	bx[4:0]						
60	bz[1]	bx[5:0]	bz[1]	bz[1]	bw[10]	bz[1]	bz[1]	bz[1]	bx[5:0]	bx[5:0]				
61														
62													bx[7:0]	
63												bx[8:0]		
64	by[3:0]	by[3:0]	by[3:0]	by[3:0]	by[3:0]	by[3:0]	by[3:0]	by[3:0]	by[3:0]	by[3:0]	bx[9:0]	bw[10]	bw[10:11]	bw[10:15]
65														
66														
67														
68				ry[3:0]	ry[3:0]									
69	ry[4:0]		ry[4:0]	bz[0]	bz[1]	ry[4:0]		ry[4:0]	ry[4:0]					
70	bz[2]	ry[5:0]	bz[2]	bz[2]	bz[2]	bz[2]	ry[5:0]	bz[2]	bz[2]	ry[5:0]				
71														
72														
73														
74				rz[3:0]	rz[3:0]									
75	rz[4:0]		ry[4]	bz[4]		rz[4:0]		rz[4:0]	rz[4:0]					
76	bz[3]	rz[5:0]	bz[3]	bz[3]	bz[3]	bz[3]	rz[5:0]	bz[3]	bz[3]	rz[5:0]				
77														
78														
79														
80														
81	d[4:0]	d[4:0]	d[4:0]	d[4:0]	d[4:0]	d[4:0]	d[4:0]	d[4:0]	d[4:0]	d[4:0]				
	10 5 5 5	7 6 6 6	11 5 4 4	11 4 5 4	11 4 4 5	9 5 5 5	8 6 5 5	8 5 6 5	8 5 5 6	6 6 6 6	10 10	11 9	12 8	16 4

This table shows the bit fields for the compressed endpoints as a function of the endpoint format, with each column specifying an encoding and each row specifying a bit field. This approach takes up 82 bits for two-region tiles and 65 bits for one-region

tiles. As an example, the first 5 bits for the one-region [16 4] encoding above (specifically the right-most column) are bits `m[4:0]`, the next 10 bits are bits `rw[9:0]`, and so on with the last 6 bits containing `bw[10:15]`.

The field names in the table above are defined as follows:

Field	Variable
m	mode
d	shape index
rw	endpt[0].A[0]
rx	endpt[0].B[0]
ry	endpt[1].A[0]
rz	endpt[1].B[0]
gw	endpt[0].A[1]
gx	endpt[0].B[1]
gy	endpt[1].A[1]
gz	endpt[1].B[1]
bw	endpt[0].A[2]
bx	endpt[0].B[2]
by	endpt[1].A[2]
bz	endpt[1].B[2]

`Endpt[i]`, where `i` is either 0 or 1, refers to the 0th or 1st set of endpoints respectively.

Sign extension for endpoint values

For two-region tiles, there are four endpoint values that can be sign extended.

`Endpt[0].A` is signed only if the format is a signed format; the other endpoints are signed only if the endpoint was transformed, or if the format is a signed format. The code below demonstrates the algorithm for extending the sign of two-region endpoint values.

```

static void sign_extend_two_region(Pattern &p, IntEndpts
endpts[NREGIONS_TWO])
{
    for (int i=0; i<NCHANNELS; ++i)
    {
        if (BC6H::FORMAT == SIGNED_F16)
            endpts[0].A[i] = SIGN_EXTEND(endpts[0].A[i], p.chan[i].prec);
        if (p.transformed || BC6H::FORMAT == SIGNED_F16)
        {
            endpts[0].B[i] = SIGN_EXTEND(endpts[0].B[i], p.chan[i].delta[0]);
            endpts[1].A[i] = SIGN_EXTEND(endpts[1].A[i], p.chan[i].delta[1]);
            endpts[1].B[i] = SIGN_EXTEND(endpts[1].B[i], p.chan[i].delta[2]);
        }
    }
}

```

For one-region tiles, the behavior is the same, only with endpt[1] removed.

syntax

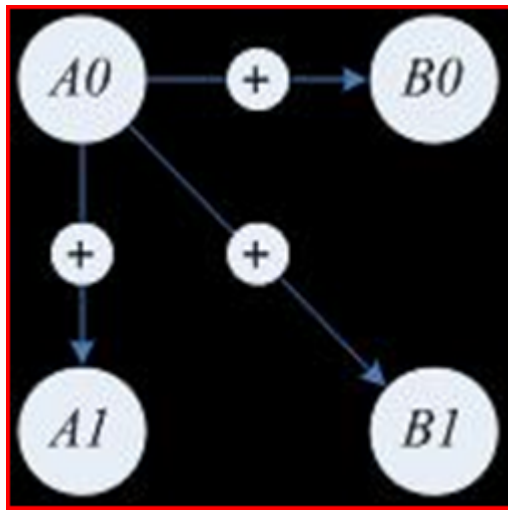
```

static void sign_extend_one_region(Pattern &p, IntEndpts
endpts[NREGIONS_ONE])
{
    for (int i=0; i<NCHANNELS; ++i)
    {
        if (BC6H::FORMAT == SIGNED_F16)
            endpts[0].A[i] = SIGN_EXTEND(endpts[0].A[i], p.chan[i].prec);
        if (p.transformed || BC6H::FORMAT == SIGNED_F16)
            endpts[0].B[i] = SIGN_EXTEND(endpts[0].B[i], p.chan[i].delta[0]);
    }
}

```

Transform inversion for endpoint values

For two-region tiles, the transform applies the inverse of the difference encoding, adding the base value at endpt[0].A to the three other entries for a total of 9 add operations. In the image below, the base value is represented as "A0" and has the highest floating point precision. "A1," "B0," and "B1" are all deltas calculated from the anchor value, and these delta values are represented with lower precision. (A0 corresponds to endpt[0].A, B0 corresponds to endpt[0].B, A1 corresponds to endpt[1].A, and B1 corresponds to endpt[1].B.)



For one-region tiles there is only one delta offset, and therefore only 3 add operations.

The decompressor must ensure that the results of the inverse transform will not overflow the precision of `endpt[0].a`. In the case of an overflow, the values resulting from the inverse transform must wrap within the same number of bits. If the precision of `A0` is "p" bits, then the transform algorithm is:

$$B0 = (B0 + A0) \& ((1 \ll p) - 1)$$

For signed formats, the results of the delta calculation must be sign extended as well. If the sign extension operation considers extending both signs, where 0 is positive and 1 is negative, then the sign extension of 0 takes care of the clamp above. Equivalently, after the clamp above, only a value of 1 (negative) needs to be sign extended.

Unquantization of color endpoints

Given the uncompressed endpoints, the next step is to perform an initial unquantization of the color endpoints. This involves three steps:

- An unquantization of the color palettes
- Interpolation of the palettes
- Unquantization finalization

Separating the unquantization process into two parts (color palette unquantization before interpolation and final unquantization after interpolation) reduces the number of multiplication operations required when compared to a full unquantization process before palette interpolation.

The code below illustrates the process for retrieving estimates of the original 16-bit color values, and then using the supplied weight values to add 6 additional color values to the palette. The same operation is performed on each channel.

syntax

```
int aWeight3[] = {0, 9, 18, 27, 37, 46, 55, 64};
int aWeight4[] = {0, 4, 9, 13, 17, 21, 26, 30, 34, 38, 43, 47, 51, 55, 60, 64};

// c1, c2: endpoints of a component
void generate_palette_unquantized(UINT8 uNumIndices, int c1, int c2, int
prec, UINT16 palette[NINDICES])
{
    int* aWeights;
    if(uNumIndices == 8)
        aWeights = aWeight3;
    else // uNumIndices == 16
        aWeights = aWeight4;

    int a = unquantize(c1, prec);
    int b = unquantize(c2, prec);

    // interpolate
    for(int i = 0; i < uNumIndices; ++i)
        palette[i] = finish_unquantize((a * (64 - aWeights[i]) + b *
aWeights[i] + 32) >> 6);
}
```

The next code sample demonstrates the interpolation process, with the following observations:

- Since the full range of color values for the **unquantize** function (below) are from -32768 to 65535, the interpolator is implemented using 17-bit signed arithmetic.
- After interpolation, the values are passed to the **finish_unquantize** function (described in the third sample in this section), which applies the final scaling.
- All hardware decompressors are required to return bit-accurate results with these functions.

syntax

```
int unquantize(int comp, int uBitsPerComp)
{
    int unq, s = 0;
    switch(BC6H::FORMAT)
    {
    case UNSIGNED_F16:
        if(uBitsPerComp >= 15)
            unq = comp;
        else if(comp == 0)
            unq = 0;
        else if(comp == ((1 << uBitsPerComp) - 1))
            unq = 0xFFFF;
        else
            unq = ((comp << 16) + 0x8000) >> uBitsPerComp;
```

```

        break;

    case SIGNED_F16:
        if(uBitsPerComp >= 16)
            unq = comp;
        else
        {
            if(comp < 0)
            {
                s = 1;
                comp = -comp;
            }

            if(comp == 0)
                unq = 0;
            else if(comp >= ((1 << (uBitsPerComp - 1)) - 1))
                unq = 0x7FFF;
            else
                unq = ((comp << 15) + 0x4000) >> (uBitsPerComp-1);

            if(s)
                unq = -unq;
        }
        break;
    }
    return unq;
}

```

finish_unquantize is called after palette interpolation. The **unquantize** function postpones the scaling by 31/32 for signed, 31/64 for unsigned. This behavior is required to get the final value into valid half range(-0x7BFF ~ 0x7BFF) after the palette interpolation is completed in order to reduce the number of necessary multiplications. **finish_unquantize** applies the final scaling and returns an **unsigned short** value that gets reinterpreted into **half**.

syntax

```

unsigned short finish_unquantize(int comp)
{
    if(BC6H::FORMAT == UNSIGNED_F16)
    {
        comp = (comp * 31) >> 6; //
        scale the magnitude by 31/64
        return (unsigned short) comp;
    }
    else // (BC6H::FORMAT == SIGNED_F16)
    {
        comp = (comp < 0) ? -((( -comp) * 31) >> 5) : (comp * 31) >> 5; //
        scale the magnitude by 31/32
        int s = 0;
        if(comp < 0)

```